

学校代号 10532

学 号 S171000897

分 类 号 TP319

密 级 公 开



湖南大学
HUNAN UNIVERSITY

硕士学位论文

PDF 论文的重编辑与处理系统的 研究与开发

学位申请人姓名 金鹏

培 养 单 位 信息科学与工程学院

导师姓名及职称 何黎刚 教授

学 科 专 业 计算机科学与技术

研 究 方 向 大数据

论文提交日期 2020 年 6 月 28 日

学校代号：10532

学 号：S171000897

密 级：公开

湖南大学硕士学位论文

PDF 论文的重编辑与处理系统的研究与 开发

学位申请人姓名： 金鹏

导师姓名及职称： 何黎刚 教授

培养单位： 信息科学与工程学院

专业名称： 计算机科学与技术

论文提交日期： 2020 年 6 月 28 日

论文答辩日期： 2020 年 6 月 28 日

答辩委员会主席： 骆嘉伟 教授

Research and development of PDF paper re-editing and processing
system

by

JIN Peng

B.E. (Yangtze University) 2017

A thesis submitted in partial satisfaction of the

Requirements for the degree of

Master of science

in

Computer Science and Technology

in the

Graduate School

of

Hunan University

Supervisor

Professor He Ligang

June, 2020

湖 南 大 学

学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果由本人承担。

作者签名：金鹏 日期：2020年7月1日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权湖南大学可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本学位论文属于

1. 保密 ☐，在 ____ 年解密后适用本授权书。

2. 不保密 ☒。

(请在以上相应方框内打“√”)

作者签名：金鹏 签字日期：2020年7月1日
导师签名：何黎刚 签字日期：2020年7月1日

摘 要

自从 PDF 文档问世以来,以其诸多的优良特性已经在学术界和工业界被广泛的使用。大量的文档资料以 PDF 格式被存档,读者只需要借助任意的 PDF 阅读器就可以方便的浏览其中所包含的信息。随着科技和文化的飞速发展,学术界已经积累了海量以 PDF 格式存档的论文,这其中绝大多数的论文内容的排版结构都是可以适配于 PC 端,在 PC 端可以渲染出良好的效果。目前移动互联网时代已经来临,各式各样的移动设备已经普及,由于移动设备不受时间和空间的约束同时具备良好的交互性等优势,越来越多的人群倾向于通过移动设备浏览和获取信息。然而 PDF 格式文件一旦被生成,其包含信息的排版格式是固定不变的,对于在 PC 端渲染良好的 PDF 学术论文,并不一定适合在屏幕尺寸较小移动设备上,这使得 PDF 学术论文在移动设备端的阅读体验大打折扣。

本文以学术界 PDF 格式论文作为研究对象,介绍了当前 PDF 文档格式的转换技术,PDF 包含信息的提取技术和渲染技术,同时对系统中涉及到的处理流程进行深入分析,设计了高效的处理模型用于优化系统的处理能力提高系统的处理效率。本文完成了对 PDF 学术论文多种格式的转化工作,并借助现有技术对 PDF 论文的内容进行分块处理,设计了基于 HTML 文档树的文本信息抽取算法和渲染样式信息的抽取算法,将提取出的本文信息按照一定的顺序结构进行重排版,同时对控制文本的渲染信息进行过滤处理,使得渲染信息能够良好的控制文本信息的渲染样式,最后将排版后的内容基于 HTML 格式作为输出结果。总的来说,本文的工作是提出一种高效的解决方案,使得 PDF 论文能够在移动设备端被渲染得更优雅。

最后对系统进行实际测试,验证了本系统可以自动化的对 PDF 论文的内容进行提取,而后对提取出的内容进行编辑排版,使得排版后的内容在移动终端能够达到较好的展示效果,本文的研究内容有利于使 PDF 文档在移动设备端呈现更加友好的渲染方式,同时对 PDF 文件文包含文本信息和非文信息的提取工作开阔了新的思路。

关键词: PDF 文档; PDF 解析; 重排版; 文本提取; 并行

Abstract

Since the PDF document came out, it has been widely used in academia and industry due to its many excellent characteristics. A large number of documents are archived in PDF format, and readers can easily browse the information contained in them with the help of any PDF reader. With the rapid development of technology and culture, the academic community has accumulated a large number of papers archived in PDF format. The layout structure of most of these papers can be adapted to the PC side, and a good rendering can be rendered on the PC side. effect. At present, the mobile Internet era has arrived, and various mobile devices have become popular. Because mobile devices are not constrained by time and space and have good interactivity and other advantages, more and more people tend to browse and obtain through mobile devices. information. However, once the PDF format file is generated, the layout format of the information contained is fixed. For a PDF academic paper that renders well on the PC, it is not necessarily suitable for mobile devices with a small screen size. This makes the PDF academic paper in the reading experience on the mobile device is greatly reduced.

This paper takes academic PDF papers as the research object, introduces the current PDF document format conversion technology, PDF contains information extraction technology and rendering technology, at the same time in-depth analysis of the processing process involved in the system, and designs an efficient processing model It is used to optimize the processing capacity of the system and improve the processing efficiency of the system. This paper completed the conversion of PDF academic papers in multiple formats, and used the existing technology to divide the content of PDF papers. Designed an HTML document tree-based text information extraction algorithm and rendering style information extraction algorithm, which will extract information in this article is re-typeset according to a certain sequence structure. At the same time, the rendering information of the control text is filtered, so that the rendering information can well control the rendering style of the text information. Finally, the typed content is based on the HTML format as the output result. In general, the work of this article is to propose an efficient solution that enables PDF papers to be rendered more elegantly on mobile devices.

Finally, we actually tested the system and verified that the system can automatically extract the content of PDF papers, and then edit and typeset the extracted content, so that the typeset content can achieve better display effects on mobile terminals. The content of the research is conducive to rendering PDF documents on mobile devices with a more friendly rendering method. At the same time, it has opened up new ideas for the extraction of PDF files containing text information and non-text information.

Keywords: PDF document; PDF parsing; re-typesetting; text extraction; parallel

目 录

学位论文原创性声明和学位论文授权使用授权书	I
摘 要	II
Abstract	III
目 录	V
插图索引	VII
附表索引	VIII
第 1 章 绪 论	1
1.1 研究背景及意义	1
1.2 国内外研究现状	2
1.3 本文主要工作	4
1.4 本文组织结构	5
第 2 章 相关文档格式的介绍	7
2.1 PDF 文档的介绍	7
2.1.1 PDF 的简介	7
2.1.2 PDF 文档对象的分析	7
2.1.3 PDF 的逻辑结构	8
2.1.4 PDF 的物理结构	9
2.2 HTML 文档的介绍	9
2.2.1 HTML 文档的简介	10
2.2.2 HTML 文档的示例	10
2.3 JSON 格式文档的介绍	11
2.3.1 JSON 的数据类型	11
2.4 本章小结	12
第 3 章 基于 HTML 文档树的内容抽取算法	13
3.1 文档格式的转换技术	13
3.1.1 转化成 WORD 格式	14
3.1.2 转化成 HTML 格式	14
3.2 PDF 文档的划分	16
3.2.1 文档划分的依据	16
3.2.2 文档划分结果的存储	17
3.3 PDF 文档信息的抽取	18
3.3.1 抽取 PDF 中的非文本信息	18

3.3.2	抽取出渲染样式	21
3.3.3	常见的网页抽取算法	23
3.3.4	抽取文本内容	23
3.4	本章小结	26
第 4 章	基于网页技术的自适应渲染方法	28
4.1	传统布局结构的分析	28
4.2	移动端网页技术的介绍	28
4.3	重构布局	28
4.4	PDF 论文的抽象	29
4.5	排版算法的分析	31
4.6	渲染效果展示	32
4.7	本章小结	35
第 5 章	并行化任务的分析与处理	36
5.1	系统后端处理流程设计	36
5.2	文件类型数据存储结构的设计	37
5.3	数据库中存储的层次关系	38
5.4	处理过程中并行化分析	39
5.5	统一任务调度方案	41
5.6	实验评估	43
5.7	本章小结	44
结 论		45
参考文献		47
致 谢		51
附录 A	攻读硕士学位期间的学术成果	52

插图索引

图 2.1 PDF 的逻辑结构	9
图 2.2 PDF 的物理结构	9
图 2.3 HTML 文档源代码	11
图 2.4 JSON 格式示例	12
图 3.1 PDF 格式展示效果	13
图 3.2 WORD 格式展示效果	14
图 3.3 HTML 格式展示效果	15
图 3.4 对 PDF 页面进行划分	17
图 3.5 PDF 页面划分结果存储的信息	17
图 3.6 从 HTML 中抽取的效果	20
图 3.7 基于图像抽取的效果	20
图 3.8 以内嵌入形式存在的渲染样式信息	21
图 3.9 LXML 解析 HTML 文档后的树型结构	22
图 3.10 PDF 转化成的 HTML 的部分源码	24
图 3.12 数据库中文本信息存储的文本信息	26
图 4.1 排版的层次结构	29
图 4.2 重构后的 HTML 源码	32
图 4.3 PDF 在移动设备上的效果	33
图 4.4 HTML 在移动设备上的效果	34
图 4.5 重构后在移动设备上的效果	34
图 4.6 屏幕尺寸相对大的渲染效果	35
图 5.1 系统后端处理流程	37
图 5.2 存储结构	37
图 5.3 PDF 在数据库中的存储关系	38
图 5.4 两层任务处理模型	39
图 5.5 线程池监控进程执行的状态	42
图 5.6 串行处理与并行处理执行的时间对比	44

附表索引

表 2.1 PDF 对象类型	8
表 3.1 JSON 格式文件中的属性及其描述信息	18
表 3.2 渲染信息的通配符及其含义	26
表 4.1 逻辑 PDF 对象的主要属性	29
表 4.2 PDF 中逻辑页的主要属性	30
表 4.3 逻辑块的主要属性	30
表 4.4 测试设备的主要参数	32
表 5.1 存储符号及其含义	38
表 5.2 任务 status 属性值及其含义	40
表 5.3 总任务的属性及其含义	40
表 5.4 服务器配置	43
表 5.5 50 份 PDF 的测试结果	43
表 5.6 多种任务处理的时间	44

第1章 绪 论

PDF 是一种便携式文档格式,由 Adobe 公司于 1993 年用作文件交换所产生和发展而来。自从 PDF 格式产生后历经多次档案格式修改,1993 年发布 PDF 1.0,1994 年发布 PDF 1.1 至 2009 年发布 PDF 1.7,此后的 PDF 格式版本交由 ISO 技术协会提供。PDF 是一种通用的文件格式,它能够保存源文档固有的字体、颜色、表格和图形。对于任何设备只要安装了 PDF 阅读器就可以加载 PDF 文档,PDF 格式也特别适合用作打印,由于 PDF 文件是以 PostScript 语言图像模型为基础,无论在何种打印机上都可以保证精确和清晰的打印效果,打印出来的文件能够完美的还原 PDF 文档原有的展示效果。PDF 格式同时也有着良好的跨平台性,可以在 UNIX 系统上创建一个 PDF 文件,然后可以在 Windows 系统上阅读。正是由于上述特点和优势,大量的学术论文使用 PDF 格式存档。随着移动互联网时代的到来,各式各样的移动设备层出不穷,人们可以通过移动设备随时随地获取网络信息资源,用户在创作学术论文时一般会借助 PC 端的一些文档工具来生成 PDF 格式的文件,其内容在布局的样式上通常是适配于 PC 端,由于移动设备屏幕尺寸一般较小导致这种布局方式可能不利于用户在移动设备上浏览。PDF 论文一旦生成其包含内容的布局方式就被固定下来,对于使用屏幕尺寸较小的移动设备的用户来说阅读起来可能不够友好,如何能够让 PDF 论文的内容在屏幕尺寸较小的设备上渲染出更好效果是一个十分有应用价值的研究性问题。

提取出 PDF 包含的内容和渲染内容的样式信息成为了关键,一旦提取出 PDF 文件上的内容和 PDF 文件上控制内容渲染的样式信息,就可以在移动设备上对提取出的信息进行重新排版和渲染,进而改变传统 PDF 文件在移动设备端的展示方式,使得 PDF 文件信息内容能够更好更便捷的被用户获取。

1.1 研究背景及意义

当前信息技术和移动技术正在飞速向前发展,移动互联网正蓬勃发展并逐渐地改变了用户使用传统 PC 端桌面软件的习惯,越来越多的移动端软件替代了 PC 端软件,尤其是当下炙热的 5G 网络技术正引领各个科技领域的变革和发展。十九大以来,国家出台的一系列政策促使移动互联网提速降费,逐渐从顶层设计至落地实施,这势必会有力促进新一轮的移动智能终端设备和移动互联网服务的飞速发展。读者不仅可以通过传统的桌面应用程序获取有效信息还可以借助移动设备随时随地更加方便的在线获取感兴趣的网络信息,这逐渐改变了千万读者的阅读习惯,事实证明移动设备获取信息更加便捷高效,传统的桌面设备严重依赖于基础环境,而移动终端通常可以便携式的随身携带不受空间时间等条件的限制。现在正处于大数据时代,数据就是价值,目前伴随深度学习和机器学习运用于实

践逐渐的成熟,可以从海量历史数据中发掘出数据的价值。自从 PDF 格式横空出世以来,已经有海量有价值的数据都以 PDF 格式的形式保存起来,当用户要获取其中的有效信息,首先必须在合适的系统环境下载并安装一个可用的 PDF 阅读器并以此打开加载 PDF 论文,用户才能开始阅览。阅读 PDF 论文就像在浏览一张图片,对于显示屏幕尺寸够大的终端来说,阅读起来十分方便,但是对于许多小屏终端尤其是智能移动手机来说由于屏幕尺寸相对较小,所以浏览起来往往需要手动放大缩小或者拖拽屏幕,这个过程对于用户来说十分的不友好,降低了用户的阅读体验。如果能把 PDF 论文的内容重新排版渲染,做到可以良好的展示在不同尺寸的屏幕上,那么对于用户来说是一种优雅的阅读体验,势必会极大的提高了用户的阅读质量,用户还可以利用零碎时间借助智能手机随时获取 PDF 格式文档包含的内容,提高了学习和工作的效率。通过智能终端移动设备将信息移动化,使得企业工作者不再受到时空限制随时随地可以参与公司会议和沟通,提高企业单位的管理能力,提高政府和企业的工作效率。因此如何高效地提取 PDF 包含的内容,并且将对提取的内容能够更人性化的在移动设备上渲染是很有实用价值且富有挑战的工作。

1.2 国内外研究现状

近些年来国内外针对 PDF 文档的研究的工作有很多,这里列举出与本课题相关的一些方向,第一类提取出 PDF 文档包含的内容信息,第二类是研究 PDF 文档格式与其他文档格式的相互转换,第三类是 PDF 阅读器的设计和实现,第四种是基于 PDF 论文包含信息进行数据挖掘工作研究。

提取 PDF 包含的文档信息是一个十分复杂的过程,PDF 文本包含的内容不仅是纯文字文本还包括图像、表格等特殊信息,同时 PDF 文档包含着控制文本对象的字体、字号等的渲染信息。文献[1]提出了一种分析 PDF 文档结构的方法,首先使开源函数库 PDFBox 来解析 PDF 文档^[43],将 PDF 包含的内容解析一系列 PDF 对象,通过分析 PDF 对象的字体、PDF 对象渲染的位置以及 PDF 对象包含的信息类别,将 PDF 页面划分成多个区域,例如论文的标题、论文的作者等,基于这种划分方式使得 PDF 页面的结构更加清晰。文献[2,3]中描述了几种算法来分析数字化文档的布局,可以将文档页面布局描述为由块组成,这些块排列在页面中并包含同一性质内容,例如文本块,图形块或表格块。文献[4]分析 PDF 文档对象的类型和属性特征,基于 PDF 对象的位置和类型进行聚合,结合 PDF 中表格标签的特征提出了一种从 PDF 文档中识别出表格区域的技术。对于排版相对固定的 PDF 文档基于文档的特征规则就可以有效的抽取出需要的信息,但是对于排版灵活复杂的 PDF 论文却不太适用,文献[5]将 SVM 技术和 PDF 对象的规则特征相结合提出了一种 PDF 论文的抽取算法,这种方法选取了 PDF 对象的一些特征,构建出训练样本集,选择出最优核函数得到 SVM 模型,基于 SVM 模型来抽取

PDF 论文中格式复杂部分的信息，但是其抽取的效率和准确率比较低。文献[6]研究的是提取药学领域中 PDF 文献包含的表格信息，基于工具 pdf2xml 首先将 PDF 格式文献转化成 XML 格式，然后对 XML 中的文本元素进行抽取分类，识别出包含表格标签的部分区域然后向下进行区域延伸，进而获取整个表格，文献[7]也是基于 XML 技术提取出 PDF 论文的信息。PDF 论文中包含着丰富的信息，为了研究学术界的最新动态和发展趋势需要对 PDF 论文的内容进行数据挖掘研究，提取出感兴趣的信息作为数据样本，文献[8]中基于开源工具 PDFBox 对 PDF 格式的学术论文进行解析，然后提取出论文的标题、作者、摘要、工作单位等信息，进而开展数据挖掘的相关研究。文献[9]以提取 PDF 文本效率为出发点，提出了一种基于自动机理论的 PDF 文本内容抽取方法来解决传统提取 PDF 内容方法效率低下的问题，例如使用 PDFBox 工具处理的效率就比较低，通过建立具有层次的关键字自动机使得提取 PDF 文档的效率有一定提升。PDF 论文中图像数据是普遍存在的，文献[10]在研究和分析了 PDF 格式的特点、PDF 生成的技术和 PDF 内部结构后，提出了一种基于图像类型，自动选择出合适的图像压缩算法处理图像，并将压缩后的图像数据写入 PDF 文件。文献[11]提出了一种基于图像格式的类型，自动地从 PDF 中提取出图像的技术，首先分析 PDF 格式中采用的图像压缩算法可以确定图像的格式类型，而后基于 PDF 文档结构，依据图像在页面中出现的顺序，对各页面中含有的图像对象进行准确定位和提取。文献[12]中使用深度学习技术来提取 PDF 文档中的表信息和图像信息，通过分析 PDF 文档的图像、表格的特征规则来训练神经网络模型。

现实工作中的需求是多样性的，通常需要对 PDF 文档进行格式转换。WORD 格式文档由于其具有良好的渲染风格和重复可编辑的特点被广泛使用，PDF 格式文档是不支持二次编辑的，文献[13]中推荐了五款 PDF 转 WORD 的工具，文献[14]中介绍了一款名为迅捷 PDF 转换器的工具，该工具已经对外提供了访问接口，可以实现 PDF 格式到 WORD 格式的转化，对于结构简单的 PDF 文档能得到较好的效果，但是对于复杂的 PDF 文档如 PDF 论文，其转化结果可能会损失部分文档信息，文献[15,16]研究了文档的转换途径，实现了将 WORD 格式文件自动转化为 PDF 格式文件。XML 格式通常用来描述数据的层次结构，比较适合文档信息的检索，文献[17]中设计了一款可以将扫描的纸质文档转化成 XML 格式的系统，其处理流程包括文档分析、文档分类、文档理解以及文档格式的转换。文献[18]基于 PDF 的规范设计了另一种将 PDF 科技论文转化成 XML 格式的系统，文献[19]中设计了一种基于 XML 的 PDF 信息的抽取系统。HTML 格式在移动终端上应用得非常普遍，文献[20,21]研究的是如何将 PDF 文档转化成 HTML 文档，主要原理是检测 PDF 上每个对象的内容以及渲染位置，然后基于 HTML 技术在生成的 HTML 文档中渲染对应的内容。文献[22]研究的是 PDF 文档在转成 HTML 时内容排版的问题，提出一种基于分割线的思想对 PDF 的每一个对象定位处理，达到转换后的 HTML 的排版样式和原 PDF 的样式一致。文献[23,24]研究的是针对 HTML

格式文档提取特定的信息,如基于 HTML 技术的特征提取出表格包含的内容。文献[25,26]介绍一些将 PDF 文件转化二进制图像文件等相关技术。PDF 论文中包含着大量有价值的信息,文献[27,28]中研究了如何从 PDF 电子文档中挖掘出有效信息,例如生成 PDF 作者的基本简历。阅读 PDF 文件需要借助于 PDF 阅读器,文献[29~31]研究了如何解析 PDF 文档和生成 PDF 阅读器的相关技术。文献[32]分析了移动互联网中 HTML5 技术的应用和研究。

由于本课题处理的流程比较复杂,为了提高系统的处理效率,借鉴了一些多任务并行处理的技术。文献[33~36]介绍了并行技术的优势和特点,文献[37~41]中研究了并行处理技术在多任务实时处理过程的应用。目前已经有许多处理 PDF 相关的工具,如一款名为 Pdf2htmlEX 的工具,它可以将 PDF 转化成 HTML 格式^[42],PDFTron 是一款可以解析 PDF 格式的技术工具,支持多种编程语言,它可以将 PDF 包含的内容解析成一系列的 PDF 元素对象^[44]。

1.3 本文主要工作

本文中研究的工作包括,转换 PDF 文件的格式,抽取出 PDF 文档内容并结构化存储,对抽取的内容进行排版和渲染。具体工作包括后台服务系统架构的设计,抽取 PDF 论文内容算法的设计,对抽取的内容进行重排版的算法设计,并行任务调度处理模型的设计等。本文对应的处理系统包括多个子处理系统,每个子系统都可以独立的运行,本文从背景调研、可行性分析、方案制定、编码实现等步骤来实现各个子系统,最后将所有模块串联起来形成一套完整的处理系统。本文调研了目前已有的 PDF 文档划分算法,基于目前已有的 PDF 论文划分技术对 PDF 论文页面内容进行划分处理,为后续解析 PDF 论文做准备工作。在对 PDF 文档进行转化的处理过程中,本文对比了许多现有的解决方案,综合考虑了效率、转化性和系统可扩展性等因素后,选择出了比较适合本课题工作的解决方案。在解析文档内容的时候进过多次尝试最终设计出了比较好的抽取算法。对于图像的处理过程中,本文对图像的分辨率进行了适当的把控,使得图像能够渲染出清晰展示效果。对于抽取出内容的存储工作,本文对文本内容基于相关规则存储进数据库中,对于非文本内容基于图像的格式存到文件系统,同时将渲染样式从文本内容中单独分离出来,存储进文件系统,这样可以动态的响应给客户端,有利于提高渲染效率。为了进一步提高系统处理效率,本文从并行处理的角度分析,充分发掘出处理过程中可并行化的流程,并设计出恰当的解决方案将部分串行处理的流程改进成并行处理,进而缩短了处理时间提高了处理效率。另外处理系统为了应对高并发请求而设计出一些可行的处理技术来解决高并发请求带来的数据不一致的问题。

抽取 PDF 中文本信息的工作通常可以借助一些 PDF 解析工具来完成,但是这些工具抽取出的信息是不带渲染样式的纯文本,而本文的目标是为了在移动设备

上能够带样式的渲染出文本。HTML 技术能够渲染出良好的效果，已经在移动端被普遍应用，所以本文使用工具将 PDF 格式转化成 HTML 格式，然后基于 HTML 文档进行后续处理。对于 PDF 中的图和表而言，其结构比较复杂，通常包含着文本和非文本信息如表格的层次结构，现有的 PDF 解析工具只能对图或表中包含的文本信息进行提取，例如表格只能提取出表格中的内部数据而且提取的信息不一定是完整的，而本文的工作是渲染出完整的样式结构，所以选择将 PDF 论文转化成图像，然后从图像上抽取出非文本信息，这样不仅可以保证图、表的文本信息完整而且渲染的结构也同样得到了保证。

本文的工作实现了提取出 PDF 论文上的内容，包括提取文本信息、渲染信息、图表信息并基于 HTML 技术对提取的内容进行重新排版渲染，使得排版后的内容能够在屏幕尺寸较小的设备终端上被优雅显示。在整个过程中包括了对文件类型的转化，HTML 格式文件的解析提取，如何提高系统处理的效率，以及重构后如何高效渲染进行了探索以及相关研究。结合文档转换技术，图像处理技术，内容分析提取技术等，设计了完整的对 PDF 文件内容的重构与渲染的解决方案。综上所述，本文对涉及到的相关技术进行多方面多角度的探索和研究，对课题中的难点提出了合适的解决方案，设计了一套 PDF 内容的抽取与渲染的处理流程，并开发出对应的 PDF 论文处理系统。

1.4 本文组织结构

本文分为 5 个章节，其中 2-5 章节是主要章节。具体如下：

第 1 章：首先介绍了该论文的研究背景和研究意义，说明了该研究的必要性，然后介绍了 PDF 文档相关技术的国内外研究现状，并且简要介绍了本文的主要研究内容。

第 2 章：介绍了一些与本文相关的概念和理论基础。从逻辑结构和物理结构两个层面对 PDF 文件结构做出了详细阐述，而后介绍了 HTML 文档格式的特点，并举出实际案例进行详细描述。最后介绍了 JSON 数据格式，并介绍了其用途和特点。

第 3 章：首先介绍了 PDF 文档和 WORD 文档、HTML 文档之间的转换技术，为提取 PDF 论文的信息做了准备工作，而后介绍了基于现有技术对 PDF 页面进行分块，并按类别划分处理，具体包括文本信息和非文本信息两个类别，最后基于 HTML 文档树和对 PDF 页面划分的结果从 HTML 中提取出 PDF 包含的文本信息和渲染信息，基于图像技术抽取出 PDF 页面的非文本信息。

第 4 章：首先分析了 PDF 论文的布局特点，阐述了 PDF 一些布局方式导致部分 PDF 论文不利于在移动设备上被浏览，并简要介绍了移动端网页技术。然后提出了一种对 PDF 页面内容进行重构和布局的方法，基于这个布局特点对 PDF 论文进行逻辑抽象，最后描述了本文的排版技术和算法，并对排版后的文档在实际

终端设备上进行了展示。

第 5 章：整个系统的处理流程比较复杂，涉及了大量的文件 IO 操作，导致处理的效率比较低，本章节主要是探讨如何提高系统的处理效率，基于并行处理的思想改进本系统的处理流程，使得处理效率得到有效提高。将 PDF 论文处理流程进行细粒度的任务化，并设计了统一任务调度管理方案，最后展示了实验数据结果来阐述各个流程的处理状况。

第2章 相关文档格式的介绍

本章主要详细描述几种文档格式的特征，介绍了 PDF 格式的历史发展，描述了 PDF 文档格式的优势和特点，从物理结构和逻辑结构层面来分析 PDF 格式文档，并举例加以详细说明，而后对 HTML 格式进行了简单的介绍，列举出了 HTML 格式的作用和优势，最后介绍了一种名为 JSON 的数据格式，这些内容为后文的章节的工作做铺垫。

2.1 PDF 文档的介绍

2.1.1 PDF 的简介

PDF 格式是由著名的 Adobe 公司研发的电子文档格式，基于 PostScript 语言基础上发展创造出来，通过特定的文档指令来描述 PDF 文件的内容。正是由于 PDF 格式文档使用的是跨平台打印控制语句来描述，使得 PDF 格式文档具有良好的跨平台性，无论是在 Windows 系统，Unix 系统还是苹果公司的 Mac OS 系统上都是通用的，并不依赖于任何底层操作系统。同时 PDF 格式文档在渲染的时候具有非常好的效果，使得用户在阅读 PDF 格式文档的时候会产生阅读纸质般的感觉。随着全球经济一体化，各行各业都在加速发展，目前由于污染导致的环境问题日益凸显。由于 PDF 文档本身具有的特殊优势，使得 PDF 格式文档在很多场景下可以替代纸质文档，既节约了资源又保护了环境，同时 PDF 格式文档一旦生成，就无法在 PDF 文档的基础上进行二次修改，所以 PDF 格式能够保证信息的有效性，目前 PDF 格式文档在各大公司和单位中成为主流的通用电子文档。

PDF 有着许多的优秀特点，其内容不同于一般的 TXT 格式文档，不仅可以包含纯文本信息同时也包含着渲染文本的样式信息，这就使得 PDF 格式文档能够满足不同场景下的需求。和图片不同，PDF 格式文档能够独立于设备和分辨率，当放大一张图像时，图像的分辨率会降低，导致图像模糊不清甚至是图像失真，对于高度集成的 PDF 文档，保存着字体、排版方式、色彩、图像和文本信息等，既包含文本信息也包含非文本信息，对 PDF 格式文件进行放大不会出现模糊的现象，同时在任意系统上将 PDF 打印出来，其内容和排版的样式完全一致。PDF 文档支持二进制编码同时也支持 7 位 ASCII 编码，可以高效的在网络上传输。PDF 格式文档自带字体信息，不会因为系统的文字缺失而不能正确显示。PDF 文档中还可以穿插音频、动画等多媒体元素，使得 PDF 格式文档能够渲染出生动活泼的效果。

2.1.2 PDF 文档对象的分析

本节对 PDF 文档的组成对象进行了分析概述，同时也从物理结构和逻辑结构

层面对 PDF 文档进行阐述，进一步加深对 PDF 文档的理解。组成 PDF 文档的基本元素单位是对象(Object)。由于 PDF 是由 PostScript 语言发展而来，因此 PDF 对象的数据类型和编程语言中的数据类型十分的相似，分为多种基本数据类型，如表 2.1 所示，其中数组(Array)是一种有序的列表结构，可以包含若干其他对象，布尔值(Boolean)、数值(Number)、名称对象(Name)为基本数据类型，字典对象(Dictionary)是一种包含若干键值对形式的对象，键为名称对象，对应的值可以为 PDF 文档内的任意对象。常见的字典对象包括 PDF 页面和字体，可以通过查阅 PDF 参考手册了解特殊属性和类型和说明。一个完整 PDF 文件是由上述对象组合而成。

表 2.1 PDF 对象类型

PDF 对象的类型	描述	示例
数组 (Array)	有序的列表	[1 2 3 4 5 6]
布尔值 (Boolean)	逻辑真/假	true
数值 (Number)	整数和实数	2.1
名称对象 (Name)	一个唯一定义的元子符号	/Type
字符串 (String)	字符串	(abc)或<abc>
字典对象 (Dictionary)	许多对象组成的表	<< /Type /Catalog /Page 3 0 R >>
流对象 (Stream)	通常表示压缩后的数据流	stream 0404040403002 endstream

2.1.3 PDF 的逻辑结构

PDF 文档包含的多种类型的对象，这些对象之间包含树型层级结构关系。树形结构使得程序可以快速定位到 PDF 的页面位置，并且读取出其包含信息。PDF 存储的逻辑结构包含一个树根对象，通过树根对象索引出四个子节点，具体包括页面节点、书签节点、线索节点和名字节点，这些节点包含的子节点一起构成了 PDF 文档的页、字体、批注等内容，具体树型结构如图 2.1 所示。PDF 文档具体包含的对象情况可能不同，大多数 PDF 文档不包含标签对象，一些也不包含索引对象。通常每个 PDF 都会包含页面对象(Page)，PDF 中的每个页面都对应一个页面对象，页面对象包含着内容流对象的引用值，具体的内容则存在于内容流对象中，PDF 文档对象既可以按照页号顺序存储，又可以任意顺序存储，顺序存储的好处就是使文件具备良好的可读性。

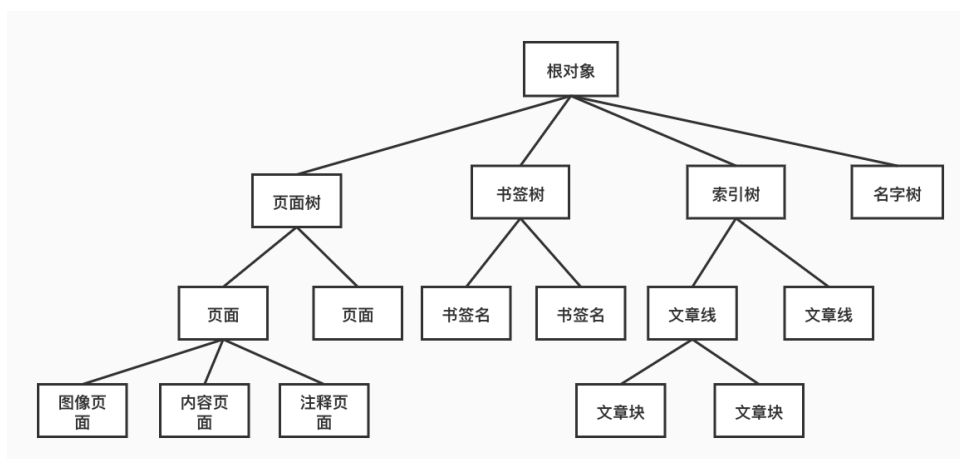


图 2.1 PDF 的逻辑结构

2.1.4 PDF 的物理结构

和 HTML 文档结构类似，PDF 也是一种结构化的文档格式，具体组成包括标记信息和数据信息，PDF 的物理结构可以分为四个部分，如图 2.2 所示，其中文件头出现在 PDF 文档的第一行，标识 PDF 文件的版本号，文件体包含着 PDF 文档的各种对象，交叉应用表包含着间接对象的信息同时存储着访问直接对象地址的索引表，文件的尾记录着交叉引用表的地址。



图 2.2 PDF 的物理结构

2.2 HTML 文档的介绍

本节从 HTML 文档的发展历程开始介绍，又分了 HTML 文档的格式和特征，通过具体的例子对 HTML 的结构化特征进行阐述。

2.2.1 HTML 文档的简介

HTML(Hyper Text Marked Language)翻译过来就是超文本标记语言,是由 Web 的发明者 Tim Berners-Lee 和同事 Daniel W. Connolly 于 1990 年创立,自从创立以来,就一直被用作万维网的信息表示语言,HTML 语言描述的文件可以通过浏览器渲染出其包含内容的效果。HTML 包括一系列语义标签。这些标签能够把网络上的文档资源统一,使分散的 Internet 资源相互关联,形成一个资源整体。HTML 文本是由 HTML 命令组成的描述性文本,HTML 命令可以是文字,图形、声音、链接、表格等。超文本是一种组织信息的方式,它通过超级链接方法将文本中的文字、图表与其他信息媒体相关联。这些相互关联的信息媒体可能在同一文本中,也可能在其他文件内,或是地理位置相距遥远的某台计算机上的文件。这种组织信息方式将分布在不同位置的信息资源用随机方式进行连接,为人们查找、检索信息提供了更加便捷的方式。

HTML 文档特性如下:

(1) 易用性: 超级文本标记语言的语义标签是确定的,只需要参考规范即可使用。

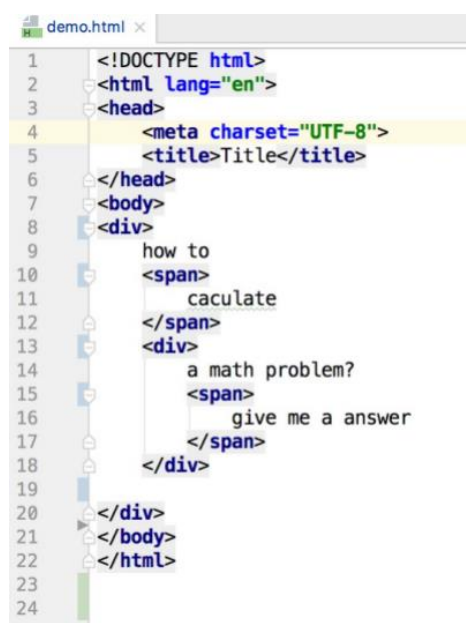
(2) 可扩展性: 超级文本标记语言的广泛应用带来了加强功能,增加标识符等要求,超级文本标记语言采取子类元素的方式,为系统扩展带来保证。

(3) 跨平台性: 无论是在 windows 系统、Unix 系统,或者是 Mac 系统上,HTML 都可以跨平台使用,这也是万维网(WWW)流行的一个重要原因。

(4) 通用性: HTML 是网络中一种简单、通用的标记语言。它支持复杂的渲染样式,例如它允许网页制作人建立文本与图片相结合的复杂页面,借助终端任何人可以在浏览器中访问到这些页面。

2.2.2 HTML 文档的示例

一个 HTML 文档源代码示例如图 2.3 所示,HTML 文档通常是以.html 或者.htm 为扩展名,用户可以使用多种文本编辑器来编辑 HTML 文件。标签<!DOCTYPE html>申明了该文档类型,<html>是文件的开头,申明了该文本是用超文本标记语言来描述的,</html>是文件的结尾,它是超文本标记语言的结束标识。<head></head>用来标识头信息,这部分是用来申明该文件被浏览器等设备加载时,呈现的某种效果,不会影响正文内容,但是会影响网页展示的效果。最常用的标签就是 meta 和 title, title 是十分重要的,它标记网页的标题,显示在浏览器的窗口栏中,标题可以被网页用作书签,被浏览器方便收藏。一个网页中的内容均被包裹在标签<body></body>之中。HTML 也支持注释,使用“<!--”和“-->”标记指明了文档注释。注释为文档提供了可读性和可维护性。



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Title</title>
6 </head>
7 <body>
8   <div>
9     how to
10    <span>
11      caculate
12    </span>
13    <div>
14      a math problem?
15      <span>
16        give me a answer
17      </span>
18    </div>
19  </div>
20 </body>
21 </html>
```

图 2.3 HTML 文档源代码

2.3 JSON 格式文档的介绍

JSON 是 JavaScript Object Notation 的缩写,这是一种轻量级的数据交换格式。它基于 ECMAScript (欧洲计算机协会制定的规范)的一个子集,JSON 不依赖于任何编程语言,它采用完全独立于编程语言的文本格式来存储和表示数据。JSON 的结构十分简单明了,具有明显的层级结构,使得 JSON 成为比较理想的数据交换格式。JSON 格式具有良好的可读性和易用性,十分易于机器对其加载和解析,许多网络服务将 JSON 格式作为数据的传输格式,有效地提升数据在网络中的传输效率。

2.3.1 JSON 的数据类型

JSON 支持多种数据类型,如字符、数字、数组、对象等,几乎可以表示任意类型的对象。JSON 是基于映射关系的方式记录数据,称为 Key-Value 键值对,一个键对应着一个值。JSON 中一个对象以“{”左括号开始,“}”右括号结束,括号中使用“,”逗号分割,形成多个键值对,每个键之后跟着一个“:”冒号,然后就是对应的值。和大多数编程语言支持的数据类型一样,JSON 也支持数组类型,每个数组中又可以包含多个对象列表。这些数据类型可以嵌套从而能够支持描述复合的数据类型。一个简单 JSON 示例如图 2.4 所示,JSON 格式的文件是以“.json”结尾,它具有明显的层级结构最外层是一对大括号,里面包含着第一层级的对象,每个对象又可以嵌套其他对象,在同一个层级中,键必须是唯一的,使用存在的键,可以索引到唯一对应的值。在传输过程中,需要考虑到传输的效率,可以将 JSON 文件压缩,去掉换行空格使得文件体积减小,提高传输效率。



```
1 {  
2   "count": 2,  
3   "people":  
4   [  
5     {  
6       "firstName": "Brett",  
7       "lastName": "McLaughlin"  
8     },  
9     {  
10      "firstName": "Jason",  
11      "lastName": "Hunter"  
12    }  
13  ]  
14 }  
15  
16  
17  
18
```

图 2.4 JSON 格式示例

2.4 本章小结

本章介绍与本文的相关技术和基础理论，是对 PDF 内容的抽取和重构实现的基础理论。简要介绍了 PDF 文档特性和结构，以及 HTML 文档格式的特点和结构，并分别列举具体示例来加以解释说明，同时也介绍了一种数据格式 JSON，它可以描述出各种复杂的数据结构。

第3章 基于 HTML 文档树的内容抽取算法

本章研究了 PDF 文档和其他几种文档格式之间的转化技术,以现有的 PDF 页面划分技术对 PDF 论文的内容按区域坐标划分成区域块,对比现有的 PDF 内容的提取技术,本文首先将 PDF 文档作格式转换,将 PDF 格式转化成 HTML 格式,而后从 HTML 文档中提取出文本内容和控制渲染文本的样式信息,对提取出的文本和渲染信息加以处理并保存,对于非文本内容,将 PDF 格式转化成图像格式后,从图像中抽取对应的非文本区域映射的图像区域。最后本章介绍几种常用的 HTML 网页抽取内容算法,分析各种算法的优势和不足,提出了适合本课题的抽取算法并加以阐述。

3.1 文档格式的转换技术

目前在学术界和工业界较为流行的文档格式为 PDF 格式和 Office 系列文档格式,包括 WORD、EXCEL、PPT 等软件产生的文件格式。目前大量学术论文以 PDF 格式存档,这些论文格式的排版布局在屏幕尺寸较大的 PC 端可以被较好的渲染,对于屏幕较小的移动终端却不一定能够良好的适配,导致用户使用移动设备浏览某些 PDF 论文十分的不便。

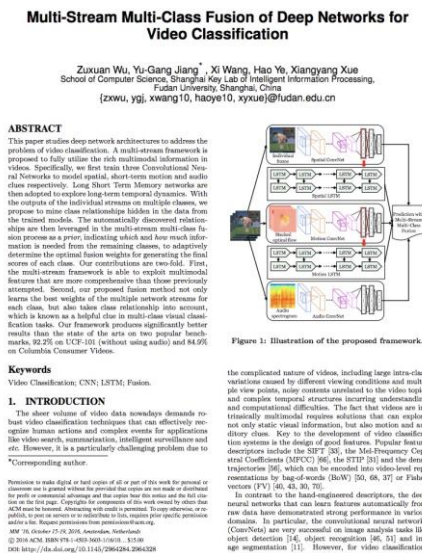


Figure 1: Illustration of the proposed framework.

图 3.1 PDF 格式展示效果

近些年来在针对文档格式转化的相关研究工作中,将 PDF 格式文档转化为其他格式文档是一项比较热门研究课题。由于 WORD 格式具有可以重复编辑的特点,所以经常需要将 PDF 格式文件和 WORD 格式文件进行相互转化,一份 PDF 格式

的文档展示效果如图 3.1 所示，这种排版风格比较高效的利用文档空间，但是由于部分内容具有左右分栏结构导致渲染的效果在屏幕较小的设备上不够友好。一旦 PDF 格式文件生成，它的样式就被固定下来，在任何尺寸的终端设备上其内容的布局方式不会发生改变，在屏幕尺寸小的设备上渲染时，PDF 页面会整体被压缩，为了浏览局部内容，用户只能通过对 PDF 页面进行整体的放缩操作，导致用户阅读效率降低。

3.1.1 转化成 WORD 格式

迅捷 PDF 转化器是一款已经面上市场的 PDF 转化器，使用迅捷 PDF 转化器对图 3.1 中的 PDF 文件进行格式转换得到的 WORD 格式如图 3.2 所示，从处理的结果来看，在内容布局上大体和原 PDF 格式文件一致，但是当对比 WORD 所包含文本信息和原 PDF 所包含文本信息时，可以发现 WORD 中包含的部分文本发生了字母的缺失，同时 WORD 包含的图片上的文字信息所在位置已经与原 PDF 中对应图片上的文字信息位置不一致。迅捷 PDF 转化器已经是一款非常成熟的工具，但是面对处理 PDF 论文这种复杂结构的文档时，难免会出现一些瑕疵。对于移动端来说，WORD 格式的展示效果依旧不尽人意。加载 WORD 格式文件，需要下载特定工具，例如 Office 办公软件或者金山公司的 WPS 办公软件，由于文件的兼容性问题，不同的办公软件加载出来的结果可能不太一致，甚至出现文件乱码，同时 WORD 文档排版的格式是固定的，如果 WORD 文档的排版风格不太适配移动端，那么用户的浏览体验也会受到影响。

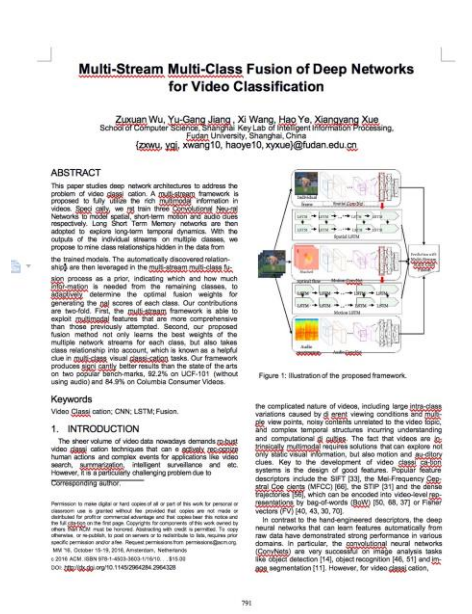


图 3.2 WORD 格式展示效果

3.1.2 转化成 HTML 格式

互联网时代只需要一个浏览器就可以访问到海量网页资源，HTML 就是网页的通用格式。移动互联网时代改变了许多用户的阅读习惯，由于移动设备的便捷

性，越来越多的用户使用移动终端的时间超过了使用 PC 端的时间，同时文件信息的存储方式也由原来的本地存储过渡到现在的云存储，在线阅读已经成为了当今时代的普遍的习惯。HTML 可以结合 CSS 技术和 JavaScript 技术，使得页面的渲染效果更加灵活^[50]，HTML 文档是无插件可交互的，用户可以在下载后离线阅读也支持在线阅读。比起 PDF 格式 HTML 格式具备更加好的兼容性，只需要任意一个浏览器就可以加载 HTML 格式文件，但是一些浏览器可能会不支持加载 PDF 格式文件。如果能将 PDF 格式文件转化成 HTML 格式，那么这个问题就得到比较好的解决。

Pdf2htmlEX 是一款开源的网络框架，它旨在将 PDF 格式文件包含的信息无损转化成 HTML 格式文件，不丢失文本信息同时保证文本渲染效果和原 PDF 文件一致。Pdf2htmlEX 利用现代 Web 技术以 HTML 格式渲染 PDF 文件，旨在提供准确的渲染结果。在操作系统中安装好 Pdf2htmlEX 之后，可以根据需要定制输入参数，例如指定生成 HTML 格式文件的存放的位置，指定需要转化 PDF 文件的页数等，带上输入参数使用 Pdf2htmlEX 提供的命令执行相关的格式转换操作。如图 3.3 所示，将图 3.1 中的 PDF 文件转化成 HTML 格式的效果，和图 3.1 的效果对比来看，转化得到的 HTML 文档在文本信息上是完整的没有任何丢失，而且渲染的样式和 PDF 格式的样式完全一致。基于此可以使用 Pdf2htmlEX 对 PDF 格式进行转化生成 HTML 格式文件，使得任意终端只需要一个浏览器就可以通过加载转化得到的 HTML 格式文档，以在线或者离线的方式浏览原 PDF 格式的信息。对于小屏幕的终端来说，尽管能够以在线的方式访问 PDF 转化后的 HTML 文档，但是由于 HTML 文档的排版布局与原 PDF 一致，导致在移动端阅读 HTML 的效果和阅读 PDF 的效果没有区别，还是不能够有效的改善阅读体验。本文的思想是将转化 PDF 得到的 HTML 文件里面的内容抽取出来，打破原有的布局方式，基于一定的规则进行重新排版布局，使得排版后的内容能够较好的渲染在屏幕尺寸较小的移动终端上。

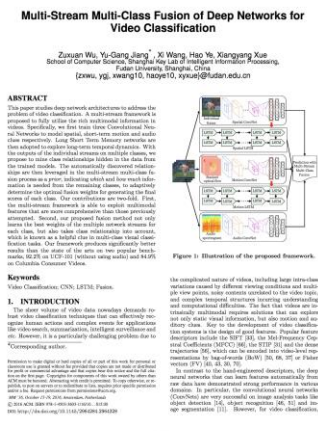


图 3.3 HTML 格式展示效果

3.2 PDF 文档的划分

PDF 格式文档可以包含丰富的内容信息，除了纯文本之外还可以插入表格和图像等非文本信息，各种类型的信息交织在一起，导致提取这些信息的工作变得十分复杂。本文的处理方法是基于现有的技术对 PDF 文档的内容进行划分，其基本思想是按照 PDF 页面中各个对象的坐标位置和文本类型对 PDF 文件内容进行分块划分，使得 PDF 的内容都被唯一的框定在一个区域块中，规定每个块的类型分为文本类型和为非文本类型中的一种，其中文本类型只包含纯文本信息，非文本信息包括包含图、表等无法用纯文本描述的信息。

3.2.1 文档划分的依据

PDF 学术论文一般来说其排版的层次感比较清晰，第一种常见的排版方式是基于左右分栏，每个分栏又是基于上下结构布局，第二种常见排版方式是基于上下结构布局，本文就是基于这种排版方式。在学术论文中，图像或者表格的所在位置通常与文本信息段落的位置有一定的区域间隔，通常文本段落信息不会环绕图像或者表格的周围，基于 PDF 论文的这种布局特点，本文使用现有技术对 PDF 论文页面的内容按照信息的类别和区域位置的特征进行分块划分。

在 PDF 文档中每个对象都有许多特征，例如文字有字号、字体、颜色，一个段落文字的字体样式大体是一致的，各章节的标题和正文的字体就是不同，而且不同段落之前会有间隔，同时对于图像对象或者表格对象都会包含着非文本元素信息，基于以上特征，本文使用了工具 PDFTron 将 PDF 以自然页为单位进行划分成块。PDFTron 是一个可以解析 PDF 的开发库，借助 PDFTron 可以构建独立的，跨平台的，可靠的业务应用程序，该应用程序可以读取，写入，打印和显示 PDF 文档。本文使用 PDFTron 对 PDF 文档进行解析，对解析的结果对象按照上述特征进行合并分类，使得每一个解析对象能够被一个矩形框所标记，记录下每个矩形的位置信息和所包含的信息类别，这里的信息类别是文本信息和非文本信息这两种划分标准，对一页中所有矩形框进行排序，排序规则基于矩形框的相对位置，按先矩形框的左上定点的坐标来排序，越靠近 PDF 页面左边沿的矩形越优先，如果靠左的位置一致，那么则比较矩形与 PDF 上边沿的距离，距离越小越优先，总的来说就是基于用户书写 PDF 论文的一般顺序进行排序。如图 3.4 所示为使用工具 PDFTron 对某一页 PDF 文档划分的结果，每一个红色矩形框代表一个被划分出来的区域块，每个矩形框有一个下标，红色下标表示该区域的是文本信息块，绿色下标表示该区域是非文本信息块。

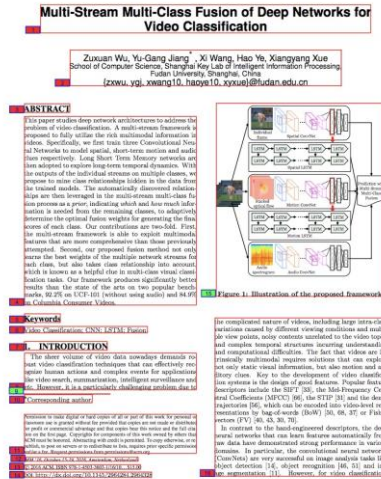


图 3.4 对 PDF 页面进行划分

3.2.2 文档划分结果的存储

本文设计了一种数据结构来保存 PDF 论文页面划分的结果，对每页 PDF 文件以像素为单位记录了页面的宽度和高度，然后用链表结构顺序的存储了所有矩形框的信息，每个矩形框记录了左上定点和右下定点的相对坐标，用来确定矩形框的位置，每个矩形框按照文本信息和非文本信息进行了类别划分。将这种结构映射成 JSON 格式后以文件的格式保存，如图 3.5 所示，描述了一页 PDF 格式文件划分后对应的 JSON 文件存储的信息，其中width表示 PDF 的宽，height表示 PDF 的高，blocks是一个列表对象包含着所有的区域对象，type表示区域的类型，bbox表示区域的坐标信息，对应的属性及其描述信息如表 3.1 所示。

```
{
  "width": 612,
  "height": 792,
  "blocks": [
    {
      "type": 0,
      "bbox": [
        76.04700000000001,
        68.27858299999999,
        533.6792270000002,
        109.13258486328107
      ]
    },
    {
      "type": 0,
      "bbox": [
        116.70499999999999,
        130.55241799999998,
        493.0274359999999,
        182.01925421142573
      ]
    },
    {
      "type": 0,
      "bbox": [
        53.79799999999999,
        204.71219999999983,
        118.23544999999999,
        3.218.68759499999987
      ]
    },
    {
      "type": 0,
      "bbox": [
        53.79799999999983,
        222.11729999999999,
        292.91048199999994,
        483.93550000000005
      ]
    },
    {
      "type": 0,
      "bbox": [
        53.79799999999983,
        493.94120000000004,
        105.18058999999998,
        5.507.91659500000001
      ]
    },
    {
      "type": 0,
      "bbox": [
        53.79799999999983,
        511.34630000000001,
        225.48793400000001,
        522.10550000000001
      ]
    },
    {
      "type": 0,
      "bbox": [
        53.79799999999983,
        532.1112,
        170.00059999999985,
        9500000001
      ]
    },
    {
      "type": 0,
      "bbox": [
        53.79799999999983,
        548.02230000000001,
        292.90397800000034,
        600.62450000000001
      ]
    },
    {
      "type": 1,
      "bbox": [
        53.29800420000001,
        604.7160034,
        173.8500042,
        605.7160034
      ]
    },
    {
      "type": 0,
      "bbox": [
        53.79800000000001,
        605.211418,
        149.26834060000007,
        618.4045
      ]
    },
    {
      "type": 0,
      "bbox": [
        53.79800000000001,
        632.360024,
        292.9224949736326,
        688.147753149414
      ]
    },
    {
      "type": 0,
      "bbox": [
        55.54200000000001,
        691.09818,
        212.56858399999999,
        698.929982
      ]
    },
    {
      "type": 0,
      "bbox": [
        53.79800000000001,
        701.889024,
        215.89379599999999,
        710.065753149414
      ]
    },
    {
      "type": 0,
      "bbox": [
        53.79800000000001,
        711.9304999999999,
        231.17145400000002,
        721.4945
      ]
    },
    {
      "type": 1,
      "bbox": [
        318.5089886617475,
        204.01353672547464,
        554.2189505026663,
        471.96695585937505
      ]
    },
    {
      "type": 0,
      "bbox": [
        316.812,
        499.57730000000004,
        555.9137016000004,
        719.5515000000001
      ]
    }
  ]
}
```

图 3.5 PDF 页面划分结果存储的信息

表 3.1 JSON 格式文件中的属性及其描述信息

序号	属性	描述
1	width	PDF 页面的高度
2	height	PDF 页面的宽度
3	blocks	区域列表
4	bbox	区域的坐标信息
5	type	区域的类型

3.3 PDF 文档信息的抽取

上一小节讨论了利用现有技术对 PDF 论文页面内容进行分块，这一节将基于上一节划分的结果对每一个分块包含的内容进行抽取。常见的 PDF 解析类库有很多，例如 PDFTron、PDFBox、PDFLib、XPDF 等，这些类库主要提供了 PDF 文件的生成技术，对于 PDF 内容提取的支持效果并不是很好，其中 PDFBox 是一款基于 Java 语言的相对优秀的解析类库，它可以把 PDF 内容解析成对象，例如文档类对象、字体类对象、页面类对象等。但是本文目的是提取出 PDF 论文的内容同时需要保证内容良好的渲染风格。上述的 PDF 类库主要是用于对内容布局简单的 PDF 文件进行比较粗糙的文本信息的提取，而且对于提取的文本信息的工作其提取的结果很可能不完整，同时基本不具备对非文本信息的提取能力。如果仅仅依靠从 PDF 文件中提取信息，是很难达到本文研究目的。基于上一小章文档的转换的技术，本文提出一种新的解决方案，对于提取 PDF 中文本信息的工作是通过将 PDF 格式论文进行 HTML 格式的转换，然后从 HTML 格式的文件中抽取出文本信息来完成的，同时可以抽取出配套 HTML 文本的渲染信息，对于非文本信息抽取的这部分工作，本文首先将 PDF 格式转化成图像格式，然后将非文本信息的位置映射到图像格式文件上，然后在图像上抽取非文本信息映射的区域，并以图像格式保存作为非文本信息的抽取结果。

3.3.1 抽取 PDF 中的非文本信息

PDF 论文的内容通常包括一些表格、图像等无法用纯文本来渲染的信息，用现有的 PDF 抽取工具无法完整高效的获取这类信息。而图像格式可以完整的表示这些非文本信息的内容及其渲染样式，本文的思路是用图像的形式来表示这些非文本区域。本文使用一款基于 Python 语言的 PDF 格式转图像格式的类库 Wand，使用 Wand 的技术将 PDF 格式的文件以页号为顺序转化成图像格式的文件^[45]，在转化前设置生成图像文件的分辨率，使得生成的图像文件比较清晰，有利于后续从中抽取非文本信息的质量，PDF 文档转换成图像的算法如算法 3.1 所示，定义的函数入口为 pdfToImg，其中参数 pdfPath 为 PDF 的路径，resultPath 为结果保存路径，resolution 为分辨率(默认设置为 300)，设置图像的分辨率并从目标路径中读取 PDF 源文件(line1)，将 PDF 文件以页为单位进行图像格式转换(line2)，遍历 PDF

的所有页，并保存生成的结果(line4-10)，循环的次数依赖于 PDF 的页数 n ，所以时间复杂度为 $O(n)$ 。

算法 3.1: PDF 转化成图像的算法(*pdfToImg(pdfPath, resultPath, resolution)*)

输入: PDF 的路径*pdfPath*，保存的路径*resultPath*，图片的分辨率*resolution*

输出: PDF 转化成图像的结果集

```

1  def pdfToImg(pdfPath, resultPath, resolution=300):
2      image_pdf = Image(filename=pdfPath, resolution=resolution)
3      image_jpg = image_pdf.convert('jpg')
4      req_image = []
5      for img in image_jpg.sequence:
6          page = Image(image=img)
7          req_image.append(page.make_blob('jpg'))
8          ff = open(resultPath, 'a')
9          ff.write(img)
10         ff.close()

```

PDF 论文转化成图像格式后，基于图 3.5 中描述的对 PDF 页面划分的结果，从图像上抽取出非文本区域，由于图像的宽与高的比值和 PDF 格式文件的宽与高的比值不一致，所以在图像上抽取非文本区域时需要进行适当转换，同时考虑到图像的精度，需要做适当的区域冗余，使得能够完整的抽取出非文本区域的信息，算法 3.2 描述的是基于算法 3.1 处理的结果对非文本信息进行抽取的算法，*deal_image*为入口函数，其中参数*blocks*为从图 3.5 的文件中获取非文本区域列表，*width*为 PDF 页面的宽度，*save_path*为保存结果的路径，*path*为要处理的目标文件路径，*redundancy*为冗余值，首先读取目标图像(line1)，计算出目标图像和 PDF 宽度的比值*ratio*(line3-4)，循环所有的非文本对象(line5)，将 PDF 文件中用矩形框框出的非文本对象区域进行映射到图像区域(line6)，将映射的区域从图像上抽取出来保存为图像格式(line7-8)，算法循环的次数基于非文本区域的数目 n ，所以时间复杂度为 $O(n)$ 。

算法 3.2: 非文本区域抽取的算法(*deal_image(p1,p2,p3,p4,p5)*)

输入: 非文本区域列表*blocks*，保存目录*save_path*，PDF 宽度*width*，图像路径*path*，冗余值*redundancy*

输出: 抽取的非文本信息

```

1  def deal_image(blocks, save_path, width, path, redundancy=2):
2      Img = Image.open(path)
3      Img_size = img.size
4      ratio = img_size[0] / width
5      for block in blocks:
6          new_bbox = [x * ratio + redundancy for x in block['bbox']]
7          region = img.crop(new_bbox)
8          region.save(save_path)

```

Pdf2htmlEX 将 PDF 格式转化为 HTML 格式时，也是基于图像格式对非文本信息进行格式转换。直接从 HTML 中抽取的图像信息如图 3.6 所示，很显然这部分图像没有包含文字信息，其原因是 Pdf2htmlEX 采用基于 HTML 元素的定位技术将文字信息覆盖到对应的位置，而本文要达到一种网页自适应的效果，因此不能使用定位技术，所以本文首先对 PDF 论文进行图像格式转换，然后基于图像文件将非文本区域包含的图像和图像上描述的文字一起抽取出来，如图 3.7 所示，图像上的文字非常完整，这种方式能够支持网页自适应渲染技术。

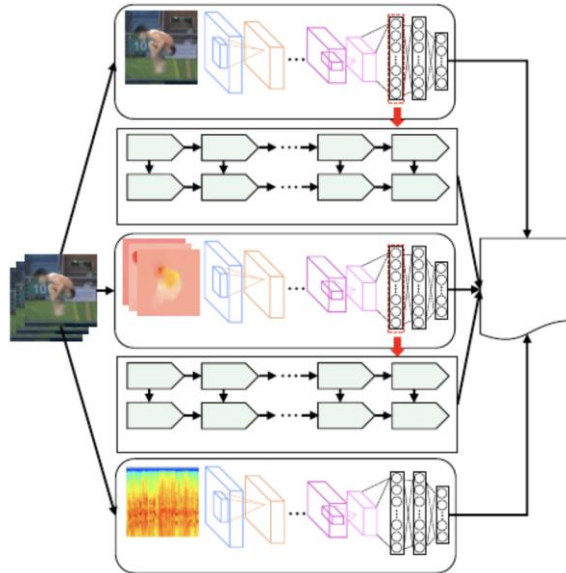


图 3.6 从 HTML 中抽取的效果

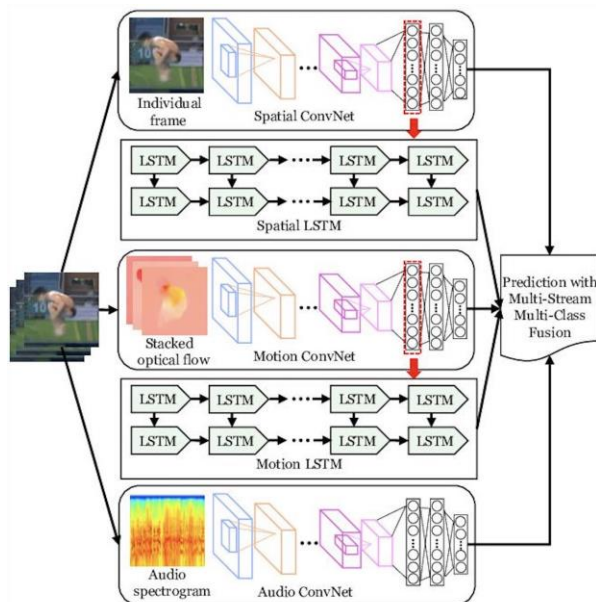


图 3.7 基于图像抽取的效果

3.3.2 抽取出渲染样式

PDF 的信息包括了文本信息和渲染文本的样式信息。PDF 论文被转化成 HTML 格式时，基于 HTML 的标签和渲染规则将 PDF 渲染的效果原样的渲染在 HTML 文件中。HTML 的渲染技术基于 CSS 技术，即层叠样式表，它的英文全称为 Cascading Style Sheets，这是一种用来控制 HTML 文件样式的计算机语言。CSS 不仅可以静态地修饰网页，还可以结合多种脚本语言动态地对网页中各个元素进行格式化的渲染处理。CSS 支持多种导入模式，可以写在 HTML 文件中，以内嵌的形式存在，也支持以外链的形式存在，基于 HTTP 协议动态的从外部服务中导入。如图 3.8 所示，以内嵌入的形式存在的渲染信息，红框的部分就是渲染该 HTML 的样式信息，基本语法是渲染信息被标签“<style></style>”所包裹。本文要抽取的 CSS 也是以内嵌入形式存在于 HTML 文件中。本文基于 HTML 的源代码，使用正则匹配技术提取出渲染的样式。LXML 是基于 Python 语言的解析库，它支持使用 XPath 技术来解析 HTML 和 XML，XPath 技术是一种用于定位 XML 和 HTML 文档中某些元素位置的技术，本文使用 LXML 解析 HTML 文件以获得树形结构元素对象，如图 3.9 所示，其中红色节点表示 HTML 中的 CSS 信息。

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>html demo</title>
</head>
<body>

  <style type="text/css">
    .main {
      font-family: "Microsoft Sans Serif";
      width: 100px;
    }
  </style>

  <div class="main">
    how to
    <span>
      caculate
    </span>
    <div>
      a math problem?
      <span>
        give me a answer
      </span>
    </div>
  </div>
</body>
</html>
```

图 3.8 以内嵌入形式存在的渲染样式信息

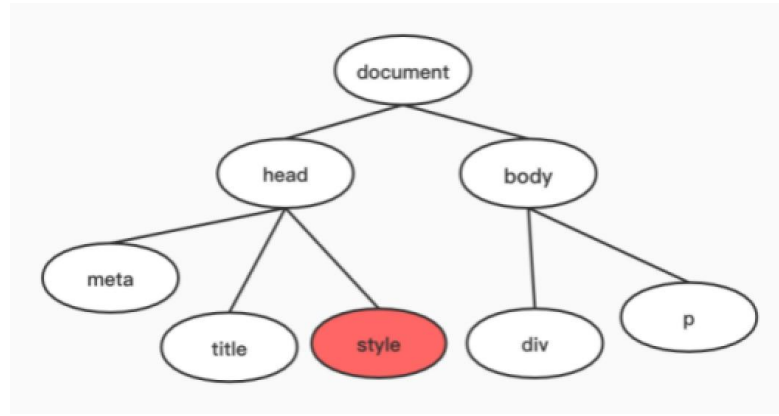


图 3.9 LXML 解析 HTML 文档后的树型结构

抽取 HTML 中渲染信息的算法如算法 3.3 所示，入口函数为 `get_style` (line1)，参数 `htmlTree` 为 HTML 的文档树对象，`path` 为存储结果的路径信息，首先创建保存结果的目录(line2)，使用 XPath 技术定位到 HTML 文档树中包含渲染信息的节点(line3)，然后将节点转化成字符串(line4)，函数 `remove_usesless_style` 的作用是过滤掉渲染信息中不被需要的部分属性(line9)，基于正则表达式的匹配技术将 `style_str` 中一些需要被移除的属性去掉，符号 (`reg1, reg2, reg3, reg4`) 分别为控制 HTML 元素的定位方式的属性、控制左边距的属性、控制底边距的属性、控制高度的属性的匹配符号(line10-17)，本文目标是达到一种自适应渲染的状态，因此以上定位元素的信息是不能被使用的，最后将过滤后的渲染信息保存起来(line5-7)。

算法 3.3: 抽取 HTML 中渲染信息的算法(`get_style(htmlTree, path)`)

输入: PDF 的路径 `pdfPath`，保存的路径 `resultPath`，图片的分辨率 `resolution`

输出: PDF 转化成图像的结果集

```

1  def get_style(htmlTree, path):
2      os.create(path)
3      style = html_tree.xpath("//style")[0]
4      style_str = etree.toString(style)
5      new_style = remove_usesless_style(style_str)
6      with open(path, 'w'):
7          file.write(new_style)
8
9  def remove_usesless_style(style_str):
10     reg1 = r'\.t.+?{position(.+);}'
11     reg2 = r'\.x.+?{left(.+);}'
12     reg3 = r'\.y.+?{bottom(.+);}'
13     reg4 = r'\.h.+?{height(.+);}'
14     reg = [reg1, reg2, reg3, reg4]
15     for item in re.findAll(reg, style_str):
16         style_str = style_str.replace(item, '')
17     return style_str

```

3.3.3 常见的网页抽取算法

目前已经有许多关于 HTML 网页抽取算法的研究，每种算法都尝试从不同的途径去解决网页内容抽取的问题。本文介绍了几种算法，并简要分析其算法处理效果。

(1) 基于 HTML 标签的抽取算法

基于 HTML 标签的抽取算法，这是一种比较容易实现的算法，首先分析出 HTML 网页的源代码结构，结合常用的标签用法和规则，定位到包含目标信息的标签，然后基于字符匹配的技术抽取出包含目标信息的标签，过滤掉标签，从而获取文本内容。这种算法对结构简单的网页有着比较好的处理效果，对于错综复杂的网页处理能力有限。

(2) 基于规则特征的正文抽取算法

基于规则特征的正文抽取算法，这种方法需要对网页的特征进行统计分析，从分析的结果中提炼出一些基本的判断规则，例如对标签中包含的文本信息数量进行统计，一般来说信息越多，越有可能包含的是正文信息。这种算法基于自定义规则的实现，局限性比较大，很多场景下会发生误判，不太适合复杂的网页。

(3) 基于机器学习的抽取算法

基于机器学习的抽取算法，该种算法对网页源代码的结构进行重构，抛弃传统对网页源码分析和抽取的方法，利用文档的特征过滤掉干扰信息，通过聚类算法将保留文本聚合成块，基于块结构抽取出正文信息，这种算法实现起来比较复杂，涉及到的处理流程比较多，最重要的是处理效果不太理想，而且处理效率也很低。

3.3.4 抽取文本内容

上一节介绍了一些提取网页正文的算法，它们的工作具有一定借鉴意义但是不能满足本课题的要求。本课题是以抽取内容且渲染内容为目的，因此本文提出了另一种提取文本信息的技术，其基本思路是从 PDF 论文转化后的 HTML 文件中提取出带标签的文本信息。一般从 HTML 中提取文本信息可以直接通过对 HTML 源代码进行分析，进而以字符串匹配的形式抽取出所需要的信息，这种方式适合于文本信息的特征比较突出，而且仅仅只是为了提取纯文本而不涉及到对提取后的内容进行渲染的工作，对于学术论文来说其格式通常比较复杂，例如字符是否加粗都代表着不同的含义，如果仅仅提取出纯文本信息则很有可能失去了文本所表达的信息，提取出的结果毫无价值，而且本文研究的是不仅提取出文本而且是保留原始的文字的渲染信息。所以本文提取出的文本信息既包含文字也包含渲染样式。PDF 转化得到的 HTML 部分源代码如图 3.10 所示，可以看出其中的文本内容和 HTML 标签交织在一起构成了 HTML 的源码，转化得到 HTML 其结构也是十分复杂的。

```

1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4 <meta charset="utf-8" />
5 <meta name="generator" content="pdf2htmlEX" />
6 <meta http-equiv="X-UA-Compatible" content="IE=edge,chrome=1" />
7 </head>
8 <body>
9 <div id="sidebar">
10 <div id="outline">
11 </div>
12 </div>
13 <div id="page-container">
14 <div id="pf1" class="pf w0 h0" data-page-no="1">
15 <div class="pc pcl w0 h0">
16 
22 <div class="t m0 x2 h2 y2 ff1 fs0 fc0 sc0 ls3 ws0"...>
25 <div class="t m0 x3 h3 y3 ff2 fs1 fc0 sc0 ls3 ws1">
26 Zuxuan W
27 <span class="-_0"></span>u, Y
28 <span class="-_1"></span>u-Gang Jiang
29 <span class="ff3 fs2 ls0 v1">*</span>, Xi W
30 <span class="-_0"></span>ang, Hao Y
31 <span class="-_2"></span>e, Xiangy
32 <span class="-_0"></span>ang Xue
33 </div>
34 <div class="t m0 x4 h4 y4 ff2 fs3 fc0 sc0 ls3 ws2"...>

```

图 3.10 PDF 转化成的 HTML 的部分源码

本文在前面章节已经讨论过对 PDF 页面的划分处理，这一小节基于划分的结果，对结果中包含的文本信息区域对应文本内容进行提取。参照 W3C 国际标准^[46]，在 HTML 格式文件中，文本内容一般是被 HTML 的标签所包裹。本文的目标是基于对 PDF 进行区域划分结果将 HTML 所包含的文本信息抽取出来，这不是简单的提取出文本块信息，而是需要根据 HTML 中各个标签元素实际渲染的相对位置来处理，如图 3.11 所示描述的是用来展示对 PDF 文件划分结果中属性 bbox 代表的信息在 PDF 渲染的页面上的实际含义。本文采用 Selenium 技术^[47]和 Chrome 浏览器及其插件^[48]来动态加载 HTML，从而获取 HTML 中的所有信息，这个过程类似于用户用 Chrome 浏览器打开 HTML 网页，浏览器在加载 HTML 页面的时候会基于 HTML 标签和 CSS 样式对该页面进行布局渲染，浏览器将 HTML 网页中的标签加载解析后生成 HTML 文档对象(后文简称 HTML 对象)，这些 HTML 对象渲染在固定的区域，可以通过解析这些 HTML 对象的属性获取其渲染的位置信息。前文已经介绍了对 PDF 页面进行了划分处理工作，记录了详细的坐标。算法 3.4 描述了如何判断一个 HTML 对象是否属于某一个区域的判别法，函数 judge 为入口函数，参数 bbox 为 PDF 上某一块区域的坐标，bottom、left 为 HTML 对象的底边距属性的值和左边距属性的值，error 为误差精度(line1)，首先获取 bbox 表示区域的坐标(line2-5)，然后对比 HTML 对象和 bbox 包含的坐标位置(line6-7)，如果 HTML 对象属于该区域则返回 True 否则返回 False (line8)，通过该算法可以识别出某一个区域包含的所有 HTML 对象。

算法 3.4: 判断 HTML 对象是否属于 bbox 表示的区域 (*judge(bbox, botytom, left, error)*)

输入: 区域坐标 bbox, HTML 对象的底边距 bottom, HTML 对象左边距 left, 误差精度 error

输出: 布尔值

```

1 def judge(bbox, bottom, left, error=2):
2     y1 = bbox[1]

```

```

3      y2 = bbox[3]
4      x1 = bbox[0]
5      x2 = bbox[2]
6      f1 = abs(y1 - error) <= bottom and bottom <= abs(y2 + error)
7      f2 = abs(x1 - error) <= left and left <= (x2 + error)
8      return f1 and f2

```

每一种 HTML 标签都包含着特殊语义，一些标签元素在渲染时会独占一行，它们被称为块级元素，无论包裹的内容是否能够沾满一行，例如标签对<p></p>可以将包裹的内容在渲染时单独成为一个段落，另一些标签如的长度依赖于包括内容的长度，不会独占一行，它们被称为行内元素。因此在对同一个bbox中的 HTML 对象进行合并时需要对块级元素特殊处理，使得同属于一个bbox区域的内容能够紧凑排列在一起，避免完整的段落出现被块级元素独占一行的现象。算法 3.5 描述了抽取文本区域包含的信息过程，函数get_content为入口函数，参数block是包含了文本区域位置信息的对象，div_list是从 HTML 中抽取的文本对象列表，包含 HTML 中所有的文本对象(line1)，首先遍历div_list的中所有div对象，判断每一个div是否属于当前block所表示的区域调用了算法 3.4 中函数(line3-4)，如果div对象属于该block那么就调用函数deal_ele来处理该div对象，从div对象中获取文本信息(line5)，首先判断该div对象是否是WebElement对象的实例(line10)，如果不是那么该对象就是纯文本对象，直接返回文本信息(line16)，否则依次遍历该div对象的子对象，对所有子对象使用函数deal_ele进行递归处理(line11-13)，最终返回出div对象包含的所有文本信息，函数get_content最终返回出属于block的所有文本对象(line6)，这样就从 HTML 中抽取出了文本信息。

算法 3.5: 抽取 PDF 文本区域的信息(*get_content (block, div_list)*)

输入: PDF 中文本区域坐标block, HTML 对象列表 div_list

输出: block包含的文本信息

```

1  def get_content(block, div_list)
2      block_text = ""
3      for div in div_list:
4          if judge(block['bbox'], div.bottom, div.left) is True:
5              block_text += deal_ele(div)
6      return block_text
7
8  def deal_ele(element):
9      child_value = ""
10     if isinstance(element, WebElement):
11         children_element = element.get_property('childNodes')
12         for child in children_element:
13             child_value += deal_div(child)
14         return child_value
15     else:

```

HTML 对象不仅包含着文本信息同时也包含着渲染样式的信息，本文的目标是对提取的文本信息进行重新排版，所以需要控制文本信息的渲染样式进行重构。一些渲染信息会干扰后续排版工作，例如控制文本元素在页面渲染的位置，本文的目标是使提取出的文本内容自适应的渲染在终端设备，因此每个文本元素没有固定的渲染位置。通过对 PDF 转化得到的 HTML 源码进行详细分析，统计了 HTML 中确定文本位置的渲染样式，然后对抽取的文本内容基于字符匹配的技术过滤掉该部分样式，如表 3.2 中所示部分渲染样式的通配符及其具体解释含义，最终将处理后的文本信息存入数据库，如图 3.12 所示为存储进数据库中的部分数据，其中 text_body 字段对应的列(红框标记的信息)就是最终存储的文本信息内容。

```

-----
| id | text_body
-----
+---+-----
-----
| 1 | <p> <span class="ff1 fs0 fc0 sc0 ws0">Multi-Stream Multi-Class Fus
cation</span></p>
-----
| 2 | <p> <span class="ff2 fs1 fc0 sc0 ws1">Zuxuan W<span class="_ _0"><
n class="_ _2"></span>e, Xiangy<span class="_ _0"></span>ang Xue</span> <
f Intelligent Inf<span class="_ _0"></span>or<span class="_ _3"></span>na
class="ff2 fs1 fc0 sc0 ws1">{zxwu, ygj, xwang10, hao<span class="_ _0">:/

```

图 3.12 数据库中文本信息存储的文本信息

表 3.2 渲染信息的通配符及其含义

序号	通配符	含义
1	x*	左边距
2	y*	底边距
3	h*	高度
4	ff*	字体样式
5	fs*	字体大小
6	t	定位方式
7	w*	宽度
8	fc*	字体颜色

3.4 本章小结

本章首先介绍了一种对 PDF 论文页面进行划分的技术，划分依据是 PDF 页面对象的渲染特征和渲染的位置，设计了一种页面数据格式用来描述划分出层次结构，并且用 JSON 格式的文件存储划分结果。而后介绍了多种提取 PDF 文本信息的方案，并提出了适合于本文的技术方案。直接从 PDF 中提取信息复杂而且容易

缺失，本文的目标是为了对 PDF 论文进行重新排版，因此不仅需要提取内容也需要对内容进行渲染，基于上述因素将 PDF 论文转化成 HTML 格式和图像格式，基于 PDF 页面的划分结构，从 HTML 中提取出文本信息和渲染信息，从图像格式中抽取非文本信息。在对 HTML 进行处理时由于论文结构的复杂性，没有采用传统对 HTML 源码解析的方法，而是采用基于浏览器动态加载 HTML 的方式，对浏览器生成的 HTML 对象树进行解析处理。总的来说本章实现了对 PDF 内容的抽取，为后续渲染排版奠定了基础。

第4章 基于网页技术的自适应渲染方法

上一章中,研究了 PDF 文件格式的转化技术和 PDF 内容的提取技术等相关处理技术,针对本文的目标,制定了合适的解决方案,首先利用 PDF 对象的特征将 PDF 论文的区域进行划分,然后基于图像技术提取出 PDF 文件中的非文本信息和基于 HTML 技术提取出 PDF 文件中的文本信息和渲染样式信息。本节内容的目标是对提取出的信息进行排版整合,基于网页技术使得排版后的效果能够在移动设备上渲染出更好的效果,提升用户阅读体验。

4.1 传统布局结构的分析

PDF 论文根据其包含内容的类型可以划分成几个类别,如文本段落、图像、表格、公式等。这些内容的布局方式通常有多种,概括来说可以分为两种。第一种将整个屏幕作为一个栏,其结构是自上至下的排列布局,这种布局方式广泛的应用在移动互联网软件上,例如各式 TXT 电子书,这种布局方式使得文本信息在字号较小的情况下也能够终端设备上渲染出清晰效果。第二种布局将整个屏幕自左至右分成多个栏,每个栏内可以基于第一种布局方式重新布局,这种布局方式广泛的应用在 PC 端,例如大量 PDF 学术论文采用这种结构,这种布局的特点是一个屏幕可以渲染出更多的信息,在屏幕尺寸较大的 PC 端有良好的渲染效果。

4.2 移动端网页技术的介绍

随着互联网技术的飞速发展,5G 时代将要来临,越来越多的用户倾向于使用移动设备访问互联网浏览信息。目前各种智能终端设备层出不穷,智能手机则是大多数用户的首选。各种移动端设备的屏幕尺寸可能不同,一个网页在不同尺寸的设备上展示出的效果会有一些差异,例如用手机打开适配于 PC 端的门户网站时,网页上的图片文字会被等比例压缩,导致图片和文字都会模糊不清,影响用户的阅读体验。目前网页常用的布局方式是响应式布局,基于屏幕的宽度尺寸,渲染出最适合的效果。因此本文基于终端设备的特点对渲染的信息进行优化处理。

4.3 重构布局

本文在前面已经讨论对 PDF 内容的提取和保存的技术,本节主要介绍如何对抽取的非文本信息和文本信息进行排版渲染。本文将 PDF 的每一页进行划分,得到多个区域,这些区域在 PDF 页面上的布局构成了完整的 PDF 页面布局。对 PDF 论文的阅读顺序进行分析,不难发现所有区域可以用阅读顺序进行串联起来,形

成类似链表结构，也就是可以将所有划分出来的区域基于用户的阅读顺序排列。如何将划分的区域映射到用户的阅读顺序成为了一个新的问题。通过分析可以得出一个结论，在正常书写规范的论文中，即 PDF 论文按照从上至下，从左至右的书写顺序，由此可以基于每个区域的位置关系来表示阅读顺序，也就是同一页 PDF 划分出来的所有区域可以基于它们的位置信息来排序。在图 3.4 中本文对每个划分的区域给出了相对位置的排版的顺序，显然这也是该 PDF 论文的阅读顺序，基于这个排序的结果本文对提取出的区域包含的内容进行排版。如图 4.1，描述了排版后 PDF 论文内部页与页和页与块之间的层次结构关系，每一个页映射着一个有序的块区域列表结构，页与页之间基于页号也形成一个有序列表结构。使用 HTML 技术将这些块列表进行处理，使得每一个块在排版后的 HTML 文件上渲染出和在原 PDF 上同样的效果。

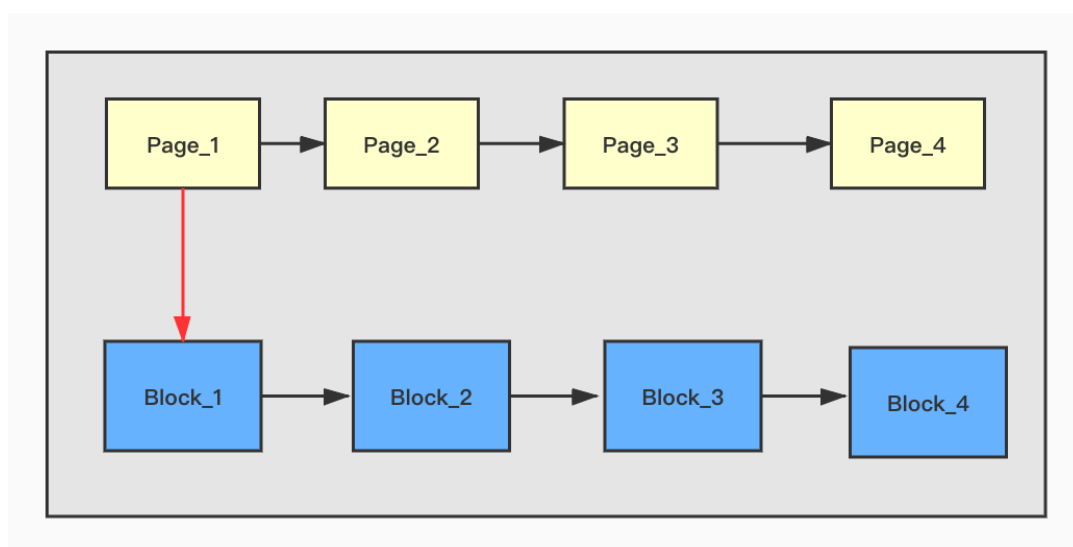


图 4.1 排版的层次结构

4.4 PDF 论文的抽象

为了高效的处理 PDF 论文，本文对 PDF 论文进行抽象处理，基于 PDF 在视觉上的层次结构，将 PDF 对象划分成三层结构，并对应地抽象成三种逻辑对象。抽象的好处在于可以用一些共同的特征来描述一类对象，方便于使用计算机语言来设计程序流程来处理这类对象。

首先是逻辑上的 PDF 对象，它是在宏观上对应着一个 PDF 论文，基于 PDF 论文的重要的共有特征来设计抽象对象，如表 4.1 所示，从物理层面的 PDF 论文中抽取并设计出一些属性，使得物理层面的 PDF 论文能够被映射到逻辑层面的 PDF 对象。

表 4.1 逻辑 PDF 对象的主要属性

编号	属性	含义
1	id	唯一标识

2	checksum	文件的 Hash 运算值
3	fileSize	文件的字节数
4	createTime	创建时间
5	status	状态值

对于 PDF 论文一般会包括多个渲染页面,本文对每个页面也进行了抽象处理,将页面的高度、页面的宽度、页面在 PDF 中的页号等特征作为抽象对象属性,构造出了一个逻辑页对象,一个逻辑 PDF 对象可以关联着多个逻辑页对象,他们是一对多的关系,可以基于逻辑 PDF 对象的唯一标识,关联到属于它的所有逻辑页对象,其主要属性值如表 4.2 所示。

表 4.2 PDF 中逻辑页的主要属性

编号	属性	含义
1	id	唯一标识
2	pageIndex	在 PDF 中的页号
3	width	页的宽度
4	height	页的高度
5	blobId	所属 PDF 对象的 id
6	status	状态值

本文在前面内容已经讨论了对 PDF 页面的划分技术,通过对 PDF 论文页的划分可以得到该页被划分成的块结构信息,将这些划分的区域块抽象成逻辑块对象,包含的属性如表 4.3 所示,这些逻辑块对象按照包含信息的类型可以分为文本块和非文本块,使用 type 属性来标识,如果是文本信息那么属性 textBody 存储着带 HTML 标签的文本内容,如果是非文本对象 textBody 存储的内容为空,但是基于该对象的其他属性可以计算出存放该区域非文本信息的路径,从而获取非文本区域的内容,属性 order 的含义是该逻辑块在所属逻辑页上的顺序编号,本文也是基于这个编号对逻辑块进行排序,最终体现在对每一页内容的排版效果。一个逻辑页对应着多个逻辑块对象,它们之间是一对多的关系,基于逻辑页的唯一标识符可以获取所关联的所有逻辑块对象。

表 4.3 逻辑块的主要属性

编号	属性	含义
1	id	唯一标识
2	type	区域块的类型
3	textBody	带 HTML 标签的文本
4	x1	区域坐标属性
5	y1	区域坐标属性
6	x2	区域坐标属性
7	y2	区域坐标属性
8	order	在所属页中的顺序编号
9	pageId	在所属页的 id

4.5 排版算法的分析

PDF 论文的每一页的所有逻辑块可以依据逻辑块的 `order` 属性进行自然排序, PDF 论文的所有页基于页号排序, 因此对于一个 PDF 论文关联的所有逻辑块对象也是有序的, 排序规则是不同页包含的逻辑块按照页号排序, 同一页中的逻辑块按照属性 `order` 排序。因此可以设计两种访问排版后内容的接口, 第一种基于逻辑页的访问, 根据请求 PDF 论文的某一页的数据进行重排版, 第二种在渲染时不考虑 PDF 原始的分页的规则, 仅仅基于逻辑块的顺序访问, 这两者的差别在于渲染的时候是否基于原 PDF 已有页号的划分规则, 但是在算法处理层面是一样的, 都是基于逻辑块进行排序组合, 算法 4.1 描述了将存储的逻辑块进行排版, 最后重构成 HTML 文件的 `body` 部分, 函数 `retype` 为入口函数, 参数 `blockList` 为某一页 PDF 论文对应的逻辑对象列表(line1), 首先基于逻辑块对象的 `order` 属性进行自然排序(line2), 对 `blockList` 进行遍历(line4), 依据 `block` 的 `type` 属性判断该 `block` 包含的信息类型是文本信息或者是非文本信息, 如果是文本信息直接合并 `block` 对象的文本属性的值(line6), 否则查询出 `block` 对象对应的非文本信息的路径, 以 HTML 技术将非文本信息以链接的形式合并进来(line7-9), 最后返回出重构后的结果(line10)。

在转换后的 HTML 文件中, 所有字体大小均以 `px` (像素) 为单位, 默认大小为 `16px`。使用 `px` 作为字体大小单位的原因是为了确保 HTML 文件具有与 PDF 文件相同的样式和布局。本文的目的是在移动屏幕上显示排版后的内容, 因此将 CSS 样式文件中的所有以 `px` 为单位的值除以 16, 然后将 `px` 更改为 `em`。`em` 是相对单位, 表示一个字宽, 其实际大小将根据用户的移动设备使用的默认字体大小而变化。最后生成 CSS 样式文件, 该文件将用于在移动终端上呈现提取的内容的样式。将重构后的 `body` 部分和 CSS 文件合并成完整的 HTML, 其部分源代码如图 4.2 所示, 这个新的 HTML 文件可以响应给移动设备端, 达到自适应的渲染效果。

算法 4.1: PDF 转化成图像的算法(`retype(p1)`)

输入: 逻辑块对象列表 `blockList`

输出: 重构后的 HTML 的 `body` 部分

```
1    def retype(blockList):
2        blockList.sort(b->b.order)
3        htmlBody = ''
4        for block in blockList:
5            if block.type == 0:
6                htmlBody += block.textBody
7            elif block.type == 1:
8                url = 'http://image/xxx'
9                htmlBody += '<p>' + '<img src = "' + url + '"/>' + '</p>'
```

[illegible]

4.6 渲染效果展示

表 4.4 测试设备的主要参数

品牌	设备	系统	分辨率	主屏尺寸
小米 8	安卓手机	Android8.1	2244x1080	5.88 英寸

本文在前面几章介绍了工具 Pdf2htmlEX 的相关概念，使用该工具可以将 PDF 格式文件转化成 HTML 格式的文件。本文将转化得到 HTML 格式文件在测试的移动端进行加载渲染，在移动设备上显示的效果如图 4.4 所示，显然渲染的效果和 PDF 源文件渲染的效果基本一致。一般来说浏览器在加载 HTML 网页时，会基于浏览器的宽度动态的排列 HTML 中的对象信息，使得布局的效果基于终端设备显示屏尺寸的不同而改变，但是这里生成的 HTML 文件渲染的布局不会随着终端设备的尺寸变化而改变。分析 HTML 源码可以看出，Pdf2htmlEX 在转化 PDF 时对 HTML 中生成各个元素标签的位置添加了相对定位的渲染效果，使得无论在

什么终端设备上渲染的效果不会变化。和阅读 PDF 一样，对于字体太小的文本内容，用户也需要对整体进行适当放缩，操作起来也是不够友好。用 HTML 代替 PDF 格式的好处在于只需要借助浏览器便可以加载内容，不受 PDF 阅读器的限制，但是仅仅只做了格式上的转换依然没有解决渲染的问题。

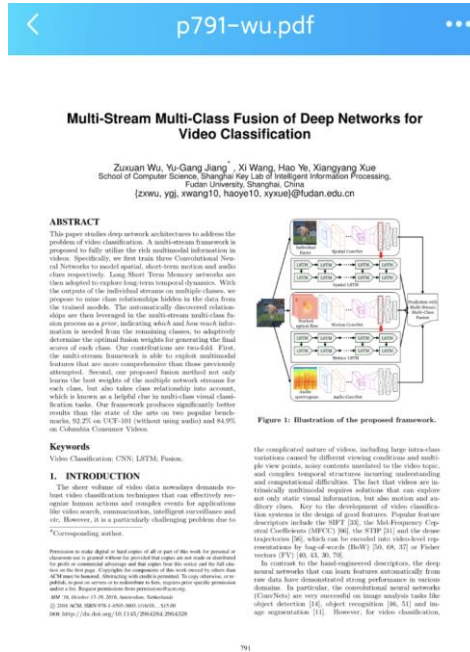


图 4.3 PDF 在移动设备上的效果

本文使用工具 Pdf2htmlEX 将 PDF 论文转化为和 HTML 格式论文，然后基于对 PDF 页面结构的划分结果，对 HTML 文件的文本内容进行抽取重构，同时从 HTML 中抽取出渲染信息进行必要的过滤处理，至于非文本信息如图像、表格等是从 PDF 论文转换的图像上抽取。最后基于 HTML 网页技术对抽取的文本信息和非文本信息进行排版。对于渲染信息则是基于用户的移动设备的默认属性进行自适应的调整。重构后的结果在测试设备上渲染的效果如图 4.5 所示，分三个屏幕展示了 PDF 文件中同样的信息，可以看出渲染的内容整体比较清晰，字体大小比较合适，同时可以基于用户习惯对文本的字体进行适当放缩，这里的放缩会使得浏览器对网页进行自适应的调整，整个内容还会在显示屏上，不同于对 PDF 文档放缩的结果可能会使得局部内容在显示屏之外。对于非文本信息，采用了网页技术使得图像以适当的大小始终会布局在显示屏的中央，如果用户想查看图片的详细内容可以点击图片单独阅览，类似于在阅读电子书。如图 4.6 所示，使用更大屏幕的设备来渲染排版后的部分展示效果，可以看出在保证字体大小合适的情况下每一行渲染的字符信息更加多，对于渲染的图片依然以比较合适的大小渲染在屏幕中央。

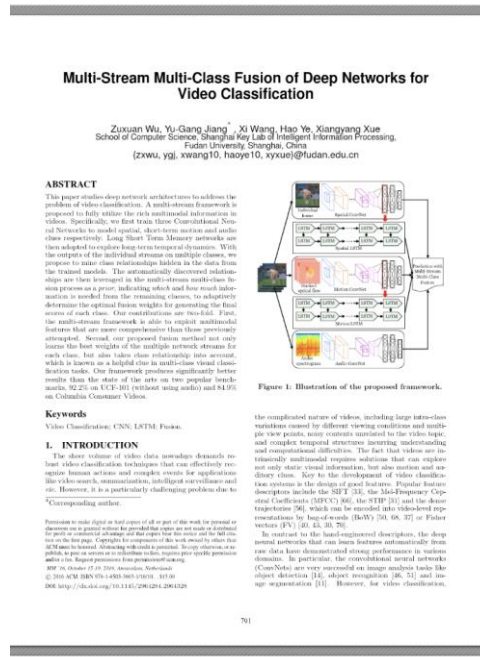


图 4.4 HTML 在移动设备上的效果

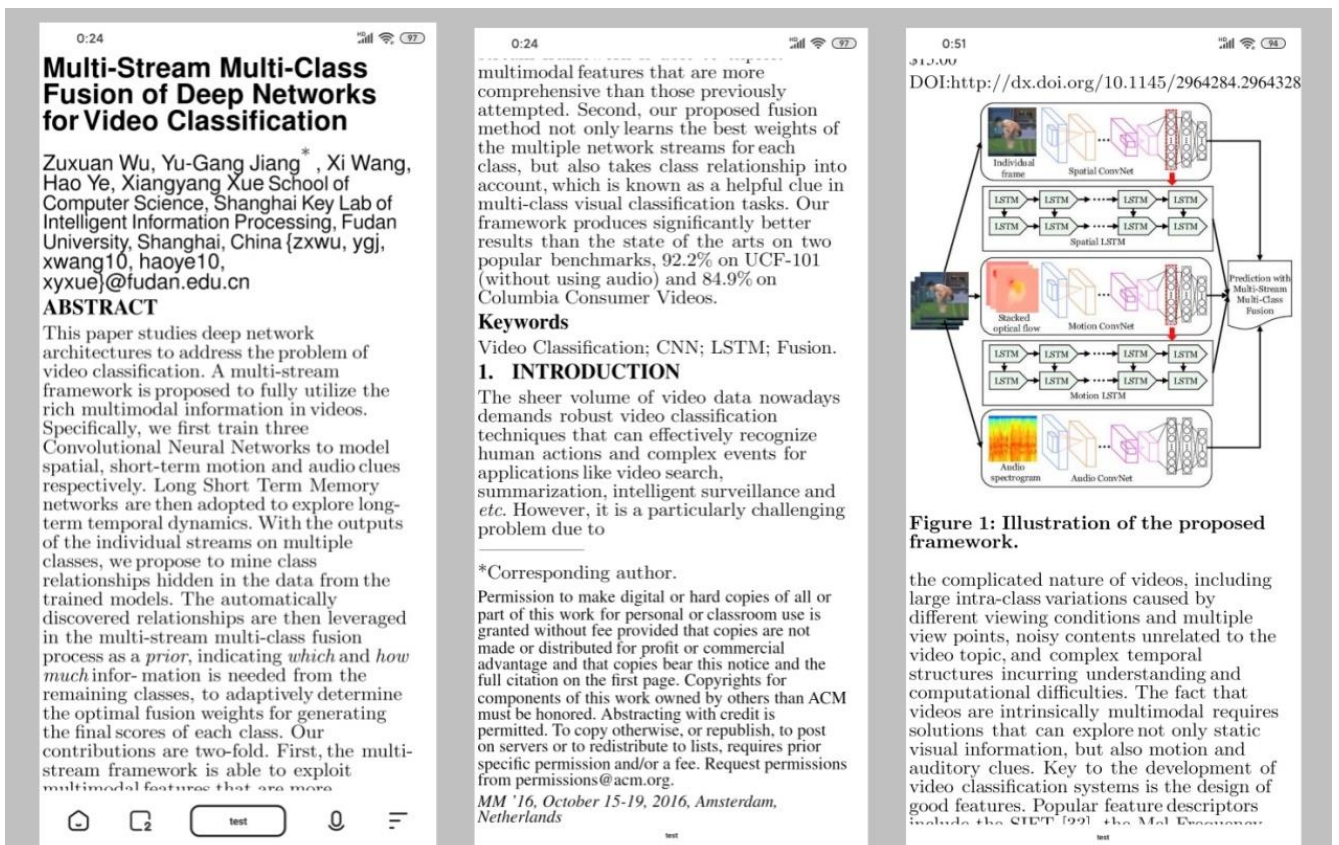


图 4.5 重构后在移动设备上的效果

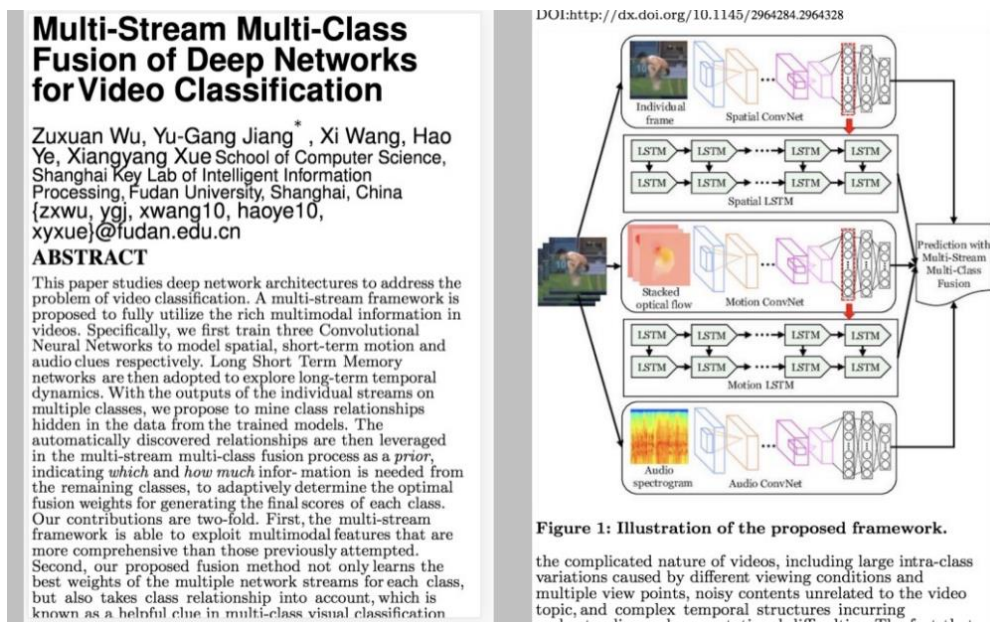


图 4.6 屏幕尺寸相对大的渲染效果

4.7 本章小结

本章主要基于网页技术对提取的信息进行重新排版布局。首先分析了 PDF 论文常见的布局方式的特点，对比了 PC 端的布局方式和移动设备端的布局方式，提出了适合于本文的布局方式，而后介绍了本文对 PDF 论文进行抽象的一种方式，详细的介绍了各个抽象对象的属性及其作用，基于这些抽象对象的特征设计了重排版算法，最后基于 HTML 技术将排版后的内容和渲染文件 CSS 进行合并组成新的自适应的 HTML 文件。最后使用小屏的移动设备对新的 HTML 文件渲染的效果进行测试，对比原始 PDF 和转化得到的 HTML 文件的渲染效果可以发现重排版后的渲染效果更适配于小屏移动端。

第5章 并行化任务的分析与处理

前面几章主要讨论研究了 PDF 论文内容提取技术和渲染技术,通过对 PDF 论文结构的分析,制定了一套完整的处理方案,涉及到使用多种技术理论来解决系统中遇到的各种问题。本章主要研究怎么将这一套完整流程中高效的串联起来达到自动化的处理效果,同时能够保证系统的稳定运行。处理过程中会产生了很多中间过度性文件,例如 PDF 转化成 HTML 格式的文件,PDF 转化成图像的文件,划分 PDF 页面后产生的 JSON 文件。为了使整个处理流程完全自动化,如何管理和存储这些文件是一个重点和难点的问题,本系统要求的是程序在处理任务的时候能够基于自定的规则,自动的索引到需要的目标文件。在本文中所有的非文件类型的数据均会被保存进数据库系统,这就需要对每个处理环节中对象进行分析,抽象出共同特征,在数据库中构造相应表结构,良好的表结构关系能使得不同对象之间的层次关系清晰,同时降低数据库的存储的冗余,提高对数据库的访问效率。本文对整个流程进行细粒度的划分,根据各环节处理数据的依赖关系,将整体处理流程拆分成一系列可单独执行的任务。由于处理流程中涉及到大量的文件转化操作,各种处理工具的调用,导致执行完成一个任务需要消耗较长的时间,使得整个系统处理的效率降低,为了更加高效处理任务,本系统基于多线程技术设计了统一任务调度管理方案,可以对每一个处理的任务进行高效的管理和自动化实时监控。为了尽可能缩短处理时间,本文分析了各个任务之间的依赖关系,对于没有依赖关系的任务,采用并行处理技术执行,提高了处理效率。

5.1 系统后端处理流程设计

本系统基于 BS 架构,采用前后端分离技术,系统前端基于 Vue 框架实现,系统后端的工作包含多个处理逻辑如图 5.1 所示,主要分为三大模块,第一个后台服务模块,该模块直接与客户端交互,主要工作包括用户的注册登陆,用户上传下载 PDF 论文,生成待处理的任务和提供给用户浏览重排版后的论文内容的接口等,该模块基于 Jfinal 框架实现。第二个模块是算法解析模块,该模块基于 Python 语言实现,使用了多种 Python 类库,主要提供多种对 PDF 论文处理的算法调用,第三个模块是定时调度与监控模块,主要工作是实现对系统中待处理任务的调度执行和监控,该模块基于 Java 语言及其相关框架实现。

系统处理过程中涉及到了大量的文件,为此本系统设计了文件管理系统,可以高效存取文件。为了提高系统的响应速度,本系统基于 Redis 设计了数据缓存,将访问量比较频繁的数据存入缓存中。对于非文件流信息,本系统一律设计出数据表,基于 Mysql 数据库存储。

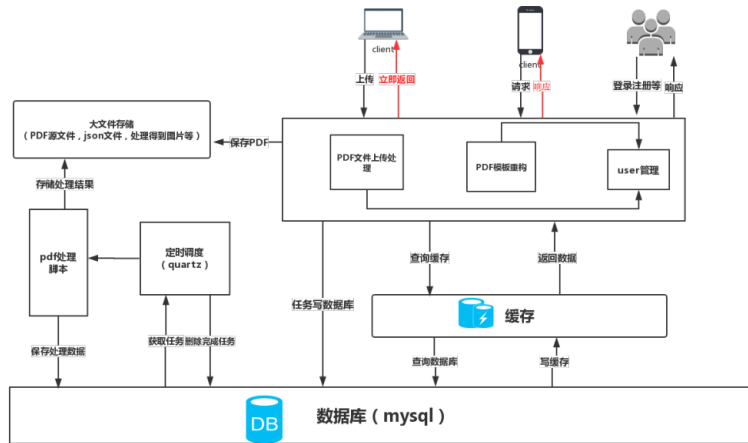


图 5.1 系统后端处理流程

5.2 文件类型数据存储结构的设计

本文中涉及到多次文件转换，包括 PDF 论文转化为图像格式，PDF 论文转化为 HTML 格式，以及非文本信息也是基于图像格式存储，和 PDF 页面结构划分结果信息的存储。为了高效的管理这些文件，本文基于树型结构设了一套存储方案，如图 5.1 所示为文件存储结构示意图，第一层为根目录入口，第二层存放的是各种文件分类入口，具体含义如表 5.1 所示，其中checksum是基于 MD5 的算法对 PDF 文件做 Hash 运算生成的值，用来唯一标识 PDF 文件。基于上述理论，本课题在实际编码时实现了通用的文件存取接口，对于任意 PDF 文件获取到其属性checksum的值就能够准确定位到相关的所有文件，降低了系统的复杂性，提高系统的可维护性。

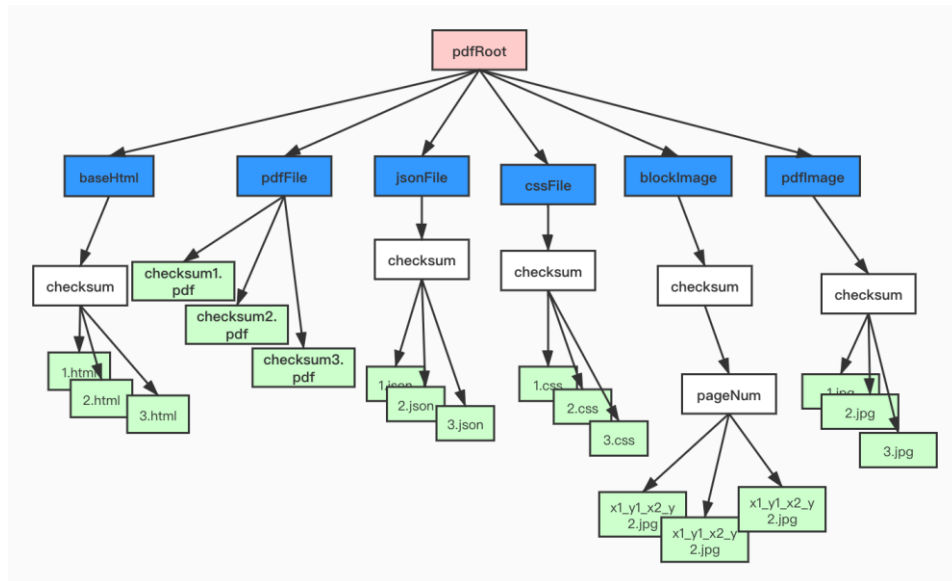


图 5.2 存储结构

表 5.1 存储符号及其含义

序号	符号	具体含义
1	pdfRoot	根目录
2	baseHtml	HTML 的目录
3	pdfFile	PDF 的目录
4	JsonFile	JSON 的目录
5	cssFile	渲染文件的目录
6	blockImage	非文本信息的目录
7	pdfImage	PDF 转化的图像的目录

5.3 数据库中存储的层次关系

本文对 PDF 文档结构进行了逻辑上的划分，使得结构更加清晰更有层次感，在实际业务中场景十分复杂，好的逻辑关系能够处理复杂的业务场景需求。在处理 PDF 论文时有可能将一份 PDF 论文上传两次，显然系统不应该去处理两次，应该在第二次上传 PDF 论文时直接返回第一次处理的结果，基于此本文将对所有上传 PDF 文档进行基于 MD5 算法的 Hash 运算，得到的运算值作为 PDF 论文的唯一标识，所以当同一篇论文被上传两次以上时基于唯一标识不会被系统处理两次。如图 5.2 所示为 PDF 论文的层级结构关系图，最上层为数据库，第二层文档(Document)与一份 PDF 论文唯一对应，第三层中描述的是页(Page)，每一份论文又包含多个页面 Page，第四层中描述的是每一页中的块(Block)，本文对每一个页面进行区域划分，每一页包含着多个区域块 Block。

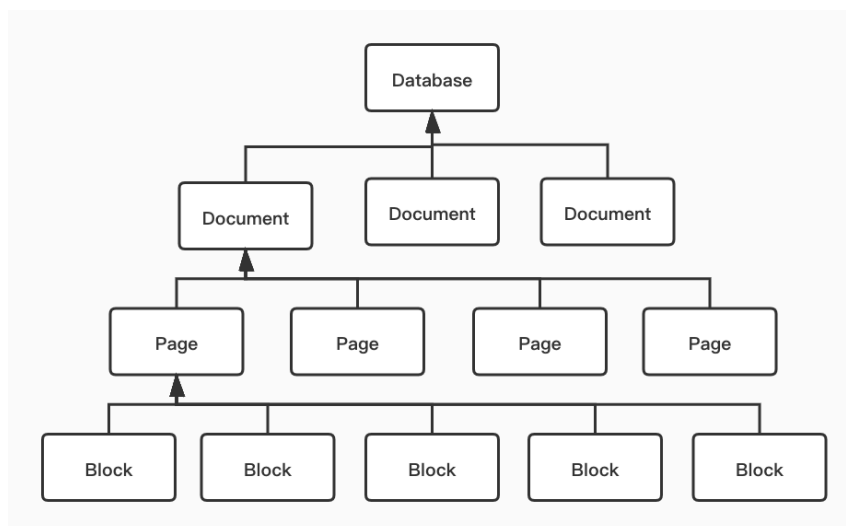


图 5.3 PDF 在数据库中的存储关系

总结上述，一个数据库中保存着多个逻辑上的 PDF 论文，每一个 PDF 论文对应着多个逻辑上的 PDF 页面，每一个页面对应着多个逻辑上的区域。通过对 PDF 论文结构进行层次划分，确立了相关数据在数据库中的逻辑存储结构。

5.4 处理过程中并行化分析

对于一个处理系统在选择串行处理技术还是并行处理技术时，需要结合实际的应用场景。通常当处理的任务之间有相互依赖关系时使用串行处理，无相互依赖关系时可以考虑使用并行处理技术，并行技术实际应用起来通常更加复杂。本文涉及到大量耗时的操作，如果直接串行执行，整个流程需要花费大量的时间，严重阻塞其他任务的执行，而且没有充分的利用服务器，造成资源的浪费。

通过对本文处理流程的分析，在对 PDF 论文进行划分和对 PDF 格式进行转换的处理操作之间并没有依赖关系，具体操作包括划分 PDF 论文的结构、将 PDF 格式转换成 HTML 格式、将 PDF 格式转化成图像格式。这些处理操作可以划分三个独立的子任务，处理它们时可以利用并行技术并行处理。在抽取文本信息依赖于对 PDF 论文的划分结果和 PDF 转化成的 HTML 文件，抽取非文本信息时依赖于 PDF 转化成的图像格式文件和对 PDF 论文划分结果，抽取渲染信息时仅依赖于 PDF 转化成的 HTML 文件，基于这些关系可以发现抽取文本信息、抽取渲染样式、抽取非文本信息之间没有依赖关系，因此这三个处理操作可以被划分成三个独立的子任务，处理它们时可以采用并行并行处理技术。通过以上分析可以抽象出一个处理模型，这个模型可以将处理流程划分成两个层次，每一层内部的任务可以并行执行，第二层依赖于第一层处理的结果，如图 5.3 所示两层任务处理模型，Level_1 层处理得到结果作为 Level_2 层的输入条件。

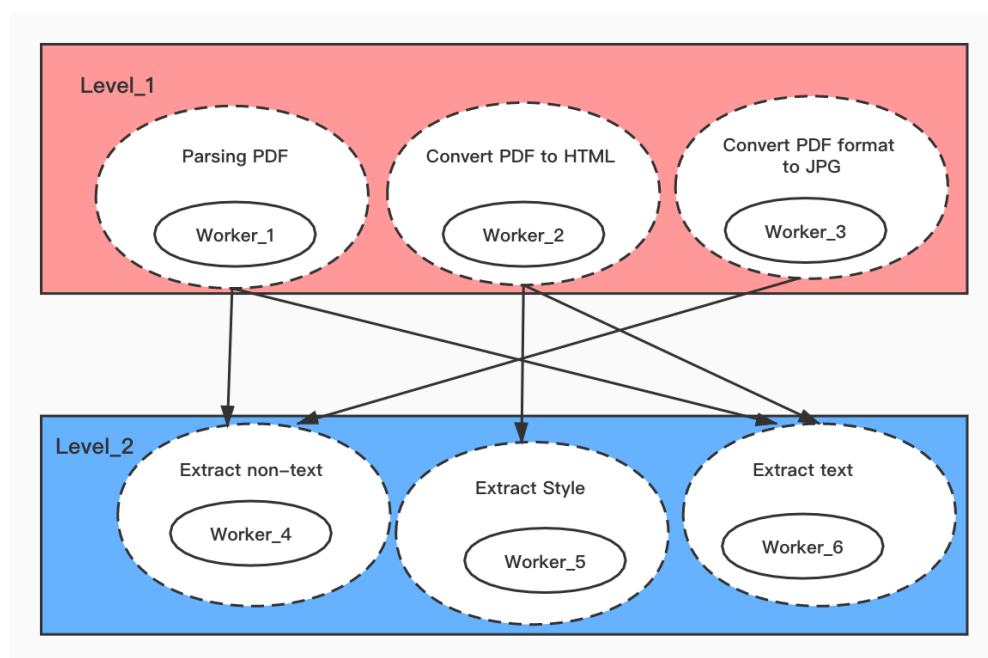


图 5.4 两层任务处理模型

依据上述模型可以将整个处理流程抽象成六大子任务，如表 5.2 所示定义每个子任务执行过程中的各种状态下的值及其含义，每一个状态值表示着子任务目前所处的状态，有了这些状态值，系统程序就可以识别出子任务目前所处的具体状态，根据状态的不同做出对应的处理对策。本文将整个流程抽象成一个总任务，

这个总任务包含多个子任务，也就是一个需要被系统处理的 PDF 论文关联一个总任务对象，总任务对象包含多种属性，其结构如表 5.3 所示，其中id表示这个总任务的索引值，blobId表示该总任务关联的 PDF 论文的索引值，status是总任务执行的状态，它不代表某一个子任务执行的情况，通过status的值系统可以识别该总任务是否执行成功或者执行异常，当某一个子任务执行异常，那么总任务对应的status会被系统修改为-1，后续不再执行该总任务包含的其他子任务，表中(f1, f2, f3, f4, f5, f6)代表着两层任务模型中抽象出来的六种子任务的状态值。通过定义子任务状态和总任务结构，可以将整个处理流程用一个总任务的对象描述出来，系统可以依据这个总任务所包含的各个子任务的执行状态来监测总任务的执行情况。基于总任务的属性在数据库中构造出用来存储总任务的表，很明显用一张数据库的表包含了六个子任务的处理状态，大大简化了系统的复杂性，节约了数据库的存储空间。

表 5.2 任务 status 属性值及其含义

状态值	含义
0	任务尚未开始执行
1	执行中
2	执行完成
-1	执行失败

表 5.3 总任务的属性及其含义

序号	字段	含义
1	id	唯一标识
2	status	执行状态
3	f1	划分 PDF 页面任务的状态
4	f2	PDF 转化成 HTML 任务的状态
5	f3	PDF 转化成图像格式任务的状态
6	f4	抽取非文本任务的状态
6	f5	抽取非文本任务的状态
7	f6	抽取渲染信息任务的状态
8	blobId	所属论文的标识

基于总任务的属性特征，本文设计一套处理子任务的逻辑流程，系统会基于总任务的 status 属性从数据库中将 status 值为 0 的总任务抽取出来，也就是将有未完成的子任务且没有执行异常的子任务所在的主任务从数据库抽取出来，作为将要处理的目标对象，分析主任务对象包含的各个子任务对象的属性值之间的关系，判断出需要执行的子任务，调用相关脚本命令执行。

算法 4.1 中描述了如何处理一个总任务，首先判断总任务mainTask的 status 状态的值为 0 表示可以执行，然后基于六个子任务的依赖关系执行相关操作，这里

Process 是一个可以开启系统进程来处理脚本命令的工具类，调用的 run 函数是一个异步非阻塞的函数，调用后直接返回，不会等待脚本执行完成，因此不会阻塞后续操作，进而实现多任务并行处理。每个处理脚本处理的结果会异步反馈给系统，系统根据各个子任务反馈的结果，在数据库中对总任务的各种状态值进行更新。更新总任务各种状态值时为避免出现线程安全问题，所有对总任务的状态值修改的操作都必须被加锁保证，本系统采用的是基于数据库的乐观锁技术实现，从而避免了多线程环境下数据不一致性的问题。

算法 4.1: 多任务并行处理算法(*dealMainTask(mainTask)*)

输入: 主任务对象 mainTask

输出: 无

```

1  def dealMainTask(mainTask):
2      if mainTask.status == 0:
3          if (mainTask.f1 == 0):
4              Process.run(task1)
5          if (mainTask.f2 == 0):
6              Process.run(task2)
7          if (mainTask.f3 == 0):
8              Process.run(task3)
9          if (mainTask.f1 == 2 && mainTask.f3 == 2):
10             Process.run(task4)
11         if (mainTask.f2 == 2):
12             Process.run(task5)
13         if (mainTask.f1 == 2 && mainTask.f2 == 2):
14             Process.run(task6)

```

5.5 统一任务调度方案

本文在上一节中将处理流程划分成六大子任务，每个子任务在满足任务执行的条件下都可以被独立执行，由于处理子任务包含着多个耗时操作而且处理任务脚本语言的不同，本文选择执行子任务时重新开启一个独立的系统进程服务，子任务在这个单独的进程中被执行。硬件资源是十分宝贵的，如果要处理的总任务很多，那么必定会产生 6 倍的子任务，这些子任务都需要操作系统开启独立的进程来处理，势必对服务器产生超负荷的压力，容易导致系统崩溃。同时任务在独立的进程被处理的时，执行的处理任务的脚本算法并不一定保证每一个任务都能够完全稳定运行，例如可能在执行程序中出现死循环的情况，导致该进程一直不能返回有效信息，使得整个处理系统发生紊乱，进程进入死循环的同时一直占有着操作系统的资源，导致系统其它服务的处理能力下降。

本文为了解决上述问题，基于线程池技术来实时监控子任务所在独立进程的

执行情况。Quartz 是一款基于 Java 语言的开源框架，采用多线程并发处理模型去实现定时任务调度执行，同时支持 Cron(用于设置周期性被执行的指令)表达式，使得 Quartz 的调度模式十分强大^[49]。总任务存储在数据库中，系统需要通过访问数据库来获取需要被执行的总任务，本系统基于 Quartz 技术实现了自动化地对数据库的轮询操作，以一个固定的周期访问数据库并获取需要被处理的主任务，使得该部分处理流程更高效。

考虑到系统资源的有限，不能无限制的开启新的进程，那么在使用 Quartz 技术轮询数据库时需要考虑到系统中现有的子任务的处理情况。引入线程池技术来控制开启独立进程的操作，线程池可以高效的利用和管理线程，避免了线程在的创建和销毁的过程中产生的大量系统开销。对于一个独立执行的进程，选择线程池中的一个空闲线程来实时监控该进程的执行情况，进程在执行子任务脚本的同时，以一定的写频率往标准输出流中写数据，这个监控线程通过系统函数获取到该进程的标注输出流，以略大于处理进程的写频率的频率从标准输出中读取数据，只要能读取数据系统就认为该进程处在正常状态，否则出现异常，如果出现异常监控线程会调用系统函数关闭处理子任务所在的进程，释放进程所占有的资源。线程池中的线程数目是可以设定的，所以这就限制了开启独立进程的数目，只有等到线程池中有空闲线程时，才能开启独立进程处理子任务，间接地控制了系统开启进程的操作，如图 5.4 所示，描述的是本系统利用线程池技术对各个进程执行情况进行监控的示意图。通过上述技术完成了对开启独立进程的控制，对处理子任务的进程执行情况进行了实时监控，使得整个系统能够更加稳定的运行。

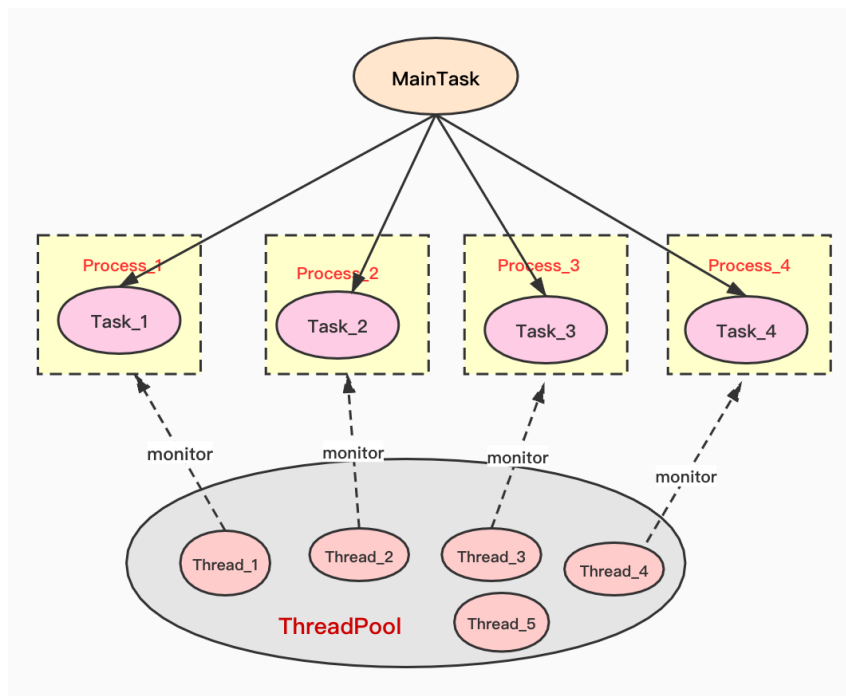


图 5.5 线程池监控进程执行的状态

5.6 实验评估

云服务器有着许多的优势，不仅为用户提供稳定的运行环境而且支持按需付费、弹性扩容等服务，基于这些优势本课题的系统也选择在云服务器上部署。世界上有着众多的云服务器供应商，例如阿里云、腾讯云、Amazon 云等。综合参考了各大云服务商的价格和提供的套餐配置，最后选择在腾讯云上租用服务器。如表 5.4 所示，列出了租用服务器的具体配置，云服务器部署的操作系统选用的是 Centos Server 7.2 LTS 64 位，物理配置包括 2 个处理核心、4G 的运行内存。

表 5.4 服务器配置

Server Configuration	
Image	Centos Server7.2Lts 64bit
CPU Count	2
CPU Topology	High Performance
CPU Model	Default
Memory	4G

本文的测试的 PDF 论文集来自于一些国际会议、期刊出版的 PDF 学术论文，涵盖了理学、工学等多个学科，从中选取了 50 篇 PDF 学术论文作为测试数据，这些 PDF 文件一共包含 530 页内容，以 PDF 的页为单位统计了每页提取的文本信息和非文本信息的正确率，如表 5.5 所示，可以看出提取渲染信息的正确率达到了 100%，其原因时渲染信息在 HTML 文档中的特征比较明显，提取文本信息正确率次之，提取非文本信息的正确率最低，其原因是非文本信息结构比较特殊，不仅包括图像、表格还包括特殊符号等，处理这部分信息的难度比较大。

表 5.5 50 份 PDF 的测试结果

测试项目	正确页的数目	准确率
提取文本信息	492	92.83%
提取渲染信息	530	100%
提取非文本信息	456	86.3%

本文对比了使用并行处理技术和串行处理技术所消耗的时间，分别抽取出多个样本的 PDF 学术论文集作为测试集，统计处理完这些文档的总时间，考虑到服务器的配置和服务器的承受能力，实验在并行处理时最多允许开启 6 个独立的处理进程。如图 5.5 所示，统计了每次并行处理和串行处理的总时间，从图像上的时间值来看，在该实验条件下，串行处理的时间明显长于并行处理的时间，在一定任务数目范围内，当处理的任务数量越多时，并行处理技术的优势越明显，这就证明力本文设计的处理模型的有效性。并行处理的效果依赖于服务器的配置，如果使用性能更好的服务器那么并行处理的优势会更加明显。

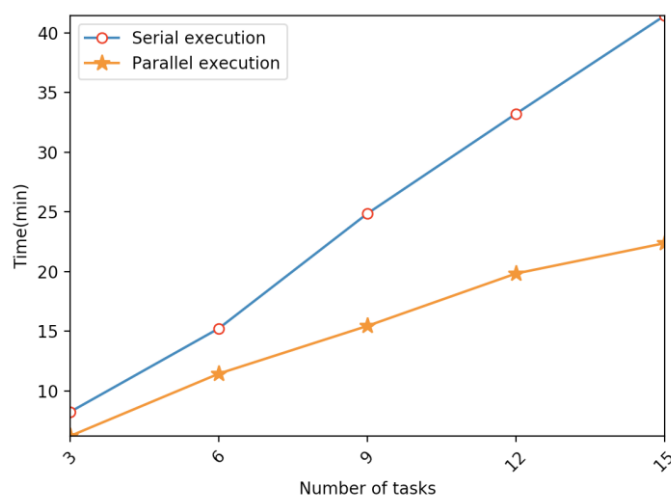


图 5.6 串行处理与并行处理执行的时间对比

实验中也统计了以 PDF 论文自然页为处理单位的多种任务处理的平均时间，实验基于并行处理模型，如表 5.6 所示，可以看出划分 PDF 页面和提取文本信息比较耗时，主要原因处理这两个任务都需要调用第三方插件进行辅助操作，例如在提取文本的工作时需要借助浏览器及其插件来动态加载和渲染 HTML 文件，这些处理任务的共同特点是都属于 IO 密集型操作，需要频繁的操作文件或者与数据库交互。

表 5.6 多种任务处理的时间

测试项目	具体工作	平均时间(秒)
JSON	划分 PDF 页面	5.34
Image	PDF 转化成 JPG	1.48
Text	抽取文本信息	3.22
NonText	抽取非文本信息	2.25
HTML	PDF 转化成 HTML	1.73
CSS	抽取渲染信息	0.52

5.7 本章小结

本章首先介绍了系统内数据的存储结构和文件的存储结构，而后对整个处理流程进行抽象，划分成六个子任务，并对每个子任务之间的依赖关系进行了详细的分析，从而制定出一套可以并行处理的模型，基于这个处理模型本文又设计了管理整个任务的统一调度管理方案，并详细介绍了利用线程池技术对每个独立进程进行实时监控管理，最后基于云服务平台部署了系统，并对系统进行了相关的性能测试。

结 论

随着互联网技术的飞速发展,智能终端设备已经成为人们在日常生活不可或缺的一部分,尤其是伴随 5G 时代即将到来,移动设备上的新技术层出不穷,由于移动设备不受时间空间的限制,越来越多的人倾向于使用智能移动设备来学习、生活、娱乐,移动设备的出现使得用户可以利用更多的时间碎片提高学习和工作的效率。在学术界的各个领域大量的学术成果基于 PDF 文档的格式被存档,对于从事学术研究的学者来说阅读大量学术论文是一项必不可少的工作,一般地阅读一篇学术论文需要在学术网站上在线阅览或者先下载到本地然后借助阅读器线下阅读。目前 PDF 是学术论文的通用文档格式,阅读论文是一项需要时间和耐心的工作,良好的阅读体验能够帮助读者快速定位到论文的核心内容。PDF 格式文档有个特点,它内容的布局方式是固定的,在不同的终端上都会展示同样的布局方式。由于 PC 端的屏幕尺寸一般比较大,一般的 PDF 格式论文能够在 PC 端优雅的展示,但是对于移动端的设备其屏幕尺寸一般小于 PC 端,那么 PDF 在移动设备渲染的效果相当于是在 PC 端的缩小版,这就导致在屏幕较小的移动设备上阅读 PDF 论文的体验很差。近些年来有许多与 PDF 格式相关的研究工作和应用,包括设计 PDF 阅读器、PDF 格式与其他格式的转换、PDF 内容提取、PDF 文档页面划分、基于 PDF 格式论文的信息挖掘等,这些研究成果对于本文的研究的内容具有重要意义。主要工作是为了能够高效而且自动化的对 PDF 论文进行解析和重排版,使得重排版得到的结果能够自适应的在移动设备终端上展示,提高用户的阅读体验。在研究过程中针对本文的目标需求和处理过程中产生的问题,提出了理论的解决方案和实际的处理算法并设计了处理系统,为了提高系统处理的效率,本文研究了相关技术并设计了高效的处理模型,而后对处理模型进行了相关测试。

本文的主要工作如下:

(1)我们借助现有的工具对 PDF 论文页面内容进行划分,划分的依据是文档对象包含的信息的类别和渲染的位置以及渲染的样式特征,使得 PDF 页面被划分成多个区域块,每一个块包含着所在的位置坐标和所包含的信息的类别,将每一块按照位置坐标排序并以 JSON 文件格式进行存储。我们为了提取出带样式的文本信息和渲染样式,首先将 PDF 格式转化成 HTML 格式,通过分析转化成的 HTML 文件源代码的结构,然后定位到渲染信息的位置并提取,同时对渲染信息中控制 HTML 元素渲染位置和控制 HTML 元素宽和高的渲染信息进行过滤处理,保存处理后的渲染信息,基于对 PDF 页面划分结果中包含的文本块的坐标,从 HTML 中抽取出每个文本块的坐标位置所包含的带 HTML 标签的文本信息,并对抽取出文本信息的 HTML 标签进行相关处理并保存。为了提取出非文本信息,我们首先将 PDF 论文转化成图像格式,将非文本块的坐标映射到对应的图像上的相关区域,后抽取出这部分区域的图像作为非文本信息。基于上述操作,我们能完成对 PDF

信息的抽取处理。

(2)我们基于网页技术对提取的信息进行重排版和布局，将排版的结果和渲染的文件组合成能够基于移动设备屏幕尺寸的自适应渲染的网页文件。首先分析了 PDF 论文常见布局方式的特点，对比了 PC 端常见的布局方式和移动设备端常见的布局方式，然后对 PDF 论文进行了分层与抽象，详细的介绍了每个层级所包含的抽象对象的属性及其作用，基于这些抽象对象的特征设计了重排版算法，最后基于 HTML 技术将排版后的内容和控制渲染的信息进行合并，组成新的能够自适应渲染的 HTML 文档，使用小屏的移动设备对排版后的效果进行测试，对比原 PDF 和原 HTML 文件的渲染效果可以发现重排版后的渲染效果更适配于小屏移动端，能明显的提升用户的阅读体验。

(3)我们对整个处理工作进行了系统性的分析，将整个处理流程进行细粒度的划分，抽象成多种子任务，利用并行处理技术提高了系统的执行效率。首先设计了系统中的数据存储结构和文件存储结构，而后对整个处理流程进行抽象，划分成六个子任务，并对每个子任务之间的依赖关系进行了详细的分析，设计出一套可以并行处理的模型，基于这个处理模型，我们又设计了管理所有子任务的统一调度管理方案，并详细介绍了利用线程池技术对每个独立进程进行实时监控管理的工作，最后基于公有云服务平台对系统进行了性能测试，结果表明使用并行技术可以明显提高系统的处理效率，验证了我们设计模型的高效性和稳定性。

工作展望：我们的工作还有一些可以改进的地方，在某些处理细节上还可以继续优化。如现有的对 PDF 页面划分处理的工具的划分精度不够理想；抽取 HTML 文档中的文本信息的算法的效率不高；我们将系统处理过程中所有关联的文件都存在一台机器上，导致存储的空间受到限制；在对任务进行并行处理时，仅用了一台云服务器，使得处理任务的效率受到硬件资源的限制；因此我们未来的工作主要集中在这几个方面：(1)优化现有解析 PDF 文档的技术，具体包括改进现有对 PDF 页面的划分算法和抽取文本信息的算法，提高算法的处理效率和准确度，在处理 PDF 时，以 PDF 页为单位对应的开启一个处理线程，通过多线程处理技术能够提升处理效率。(2)本系统处理过程涉及大量的文件，我们应该使用分布式存储技术来管理文件，分布式存储系统可以提升系统对文件的吞吐量，同时提高目标文件的索引效率。(3)我们可以将系统部署到多台服务器上，采用集群架构，例如每一种任务对应一个处理集群，又或者基于一些负载均衡的策略将需要处理的任务分发到具体的服务器上，这样可以显著提升系统处理的吞吐量。

参考文献

- [1] Hassan, Tamir. Object-level document analysis of PDF files. in Proceedings of the 9th ACM symposium on Document engineering, 2009, pp. 47-55.
- [2] Tuarob S, Bhatia S, Mitra P, et al. Automatic detection of pseudocodes in scholarly documents using machine learning. in 2013 12th International Conference on Document Analysis and Recognition, 2013, pp.738-742.
- [3] Diem M, Kleber F, Sablatnig R. Text classification and document layout analysis of paper fragments. in 2011 International Conference on Document Analysis and Recognition, 2011, pp. 854-858.
- [4] Hassan T, Baumgartner R. Table recognition and understanding from pdf files. in Ninth International Conference on Document Analysis and Recognition, 2007, pp.1143-1147.
- [5] 李雪驹, 王智广, 鲁强. 一种规则与 SVM 结合的论文抽取方法[J]. 计算机技术与发展, 2017, 27(10):24-29.
- [6] 窦方坤, 曹皓伟, 徐建良. 基于文本元素的 PDF 表格区域识别方法研究[J]. 软件导刊, 2020, 19(01):113-116.
- [7] 宋艳娟, 张文德. 基于 XML 的 PDF 文档信息抽取系统的研究[J]. 现代图书情报技术, 2005(9):10-13.
- [8] 牛永洁, 薛苏琴. 基于 PDFBox 抽取学术论文信息的实现[J]. 计算机技术与发展, 2014, 24(12): 61-63.
- [9] 王晓娟, 谭建龙, 刘燕兵等. 基于自动机理论的 PDF 文本内容抽取[J]. 计算机应用, 2012, 32(09):2491-2495.
- [10] 赵玉琴. 图像转 PDF 的问题与解决方法[J]. 档案与建设, 2012 (12): 60-60.
- [11] 陈云榕, 刘立柱, 叶晗. PDF 文档中 JPEG 图像的自动提取技术研究[J]. 信息工程大学学报, 2007, 8(2): 213-216.
- [12] Hansen M, Pomp A, Erki K, et al. Data-Driven Recognition and Extraction of PDF Document Elements. Technologies, 2019, 7(3): 65.
- [13] 何翌. 5 款 PDF 转 Word 转换器推荐[J]. 计算机与网络, 2017, 43(6): 37-37.
- [14] 边世雄. 迅捷 PDF 转换器可将 PDF 转换成 word[J]. 计算机与网络, 2017, 1.
- [15] 孟祥丽, 张照余. 文本文件归档格式及转换途径研究[J]. 档案时空, 2017 (10): 14-17.
- [16] 扈小燕, 刘培洵, 刘力强. 将 Word 文档自动转换成 PDF 格式的编程实现

- [J]. 计算机与现代化, 2012 (2): 187-189.
- [17] Altamura O, Esposito F, Malerba D. Transforming paper documents into XML format with WISDOM++. *International Journal on Document Analysis and Recognition*, 2001, 4(1): 2-17.
- [18] Déjean H, Meunier J L. A system for converting PDF documents into structured XML format. in *International Workshop on Document Analysis Systems*. Springer, Berlin, Heidelberg, 2006, pp.129-140.
- [19] 宋艳娟, 张文德. 基于 XML 的 PDF 文档信息抽取系统的研究[J]. 现代图书情报技术, 2005 (9): 10-13.
- [20] LIN Q, LI J. A Study of Text Rearrang in Conversion of PDF Documents into HTML. *Computer and Information Technology*, 2014, pp.3-10.
- [21] Jiang D, Yang X. Converting PDF to HTML approach based on Text Detection. in *Proceedings of the 2nd international conference on interaction sciences: Information technology, culture and human*. 2009, pp.982-985.
- [22] 林青, 李健. PDF 文档 HTML 化中文本重排问题研究[J]. 电脑与信息技术, 2014, 22(03): 27-30.
- [23] Mirończuk M M. The BigGrams: the semi-supervised information extraction system from HTML: an improvement in the wrapper induction. *Knowledge and Information Systems*, 2018, 54(3): 711-776.
- [24] Rastan R, Paik H Y, Shepherd J. Texus: A unified framework for extracting and understanding tables in pdf documents. *Information Processing & Management*, 2019, 56(3): 895-918.
- [25] 顾韦华. 基于 ImageMagick 和 Magickwand 的 Web 图像处理[J]. 电脑编程技巧与维护, 2010(16): 107-109.
- [26] Suzuki M, Terada Y, Kanahori T, et al. New Tools to Convert PDF Math Contents into Accessible e-Books Efficiently. *Studies in health technology and informatics*, 2015, 217: 1060-1064.
- [27] 于江德, 樊孝忠, 尹继豪, 等. 基于隐马尔可夫模型的中文科研论文信息抽取[J]. 计算机工程, 2007, 33(19): 190-192.
- [28] Zhou C, Li S. Research of information extraction algorithm based on hidden markov model. in *The 2nd International Conference on Information Science and Engineering*. IEEE, 2010, pp.1-4.
- [29] 李强, 刘时进. PDF 阅读器的设计与实现[J]. 计算机工程与设计, 2010 (7): 1635-1638.
- [30] 陈卓, 张荣, 郭立君, 等. PDF 电子签章控件关键技术研究 with 实现[J]. 移动

- 通信, 2019 (5): 18.
- [31] 赵刚, 于悦, 黄敏桓, 等. PDF 阅读器字体解析引擎的测试方法[J]. 2018.
- [32] 李慧云, 杨新章, 胡文胜, 等. 基于 HTML5 的移动互联网应用转换技术[J]. 电信科学, 2013, 29(5): 28-32.
- [33] Burns B, Grant B, Oppenheimer D, et al. Borg, omega, and kubernetes. Queue, 2016, 14(1): 70-93.
- [34] 王小岗, 王小明, 陈庆新, 等. 具有随机返工且可中断的设计任务调度[J]. 工业工程, 2019, 22(5): 59.
- [35] Tang D, Zheng L, Li Z, et al. Re-engineering of the design process for concurrent engineering. Computers & Industrial Engineering, 2000, 38(4): 479-491.
- [36] 易肃汶, 戴紫彬, 李伟. 面向 IDEA 算法的多引擎高速并行化研究与设计[J]. 计算机工程与设计, 2017, 38(1): 37-41.
- [37] Chen T, Srinath S, Batten C, et al. An architectural framework for accelerating dynamic parallel algorithms on reconfigurable hardware. in 2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). IEEE, 2018, pp.55-67.
- [38] 时小虎. 实时并行处理技术在指控系统中的应用. 见: 中国指挥与控制学会.2014 第二届中国指挥控制大会论文集(上).中国指挥与控制学会:中国指挥与控制学会, 2014, 585-591.
- [39] 陈泽玮, 杨茂林, 雷航, 等. SET-MRTS: 一种多处理器实时系统可调度性实验平台[J]. 计算机应用, 2017, 37(5): 1270-1275.
- [40] Panda S K, Gupta I, Jana P K. Task scheduling algorithms for multi-cloud systems: allocation-aware approach. Information Systems Frontiers, 2019, 21(2): 241-259.
- [41] Glushkova D, Jovanovic P, Abelló A. Mapreduce performance model for Hadoop 2. x. Information Systems, 2019, 79: 32-43.
- [42] Wang L, Liu W. Online publishing via pdf2htmlEX[J]. The Communications of the TeX Users Group, 2013, 34(3): 313-324.
- [43] Yang H, Aguirre C A, Maria F, et al. Pipelines for Procedural Information Extraction from Scientific Literature: Towards Recipes using Machine Learning and Data Science. in 2019 International Conference on Document Analysis and Recognition Workshops (ICDARW). IEEE, 2019, pp.41-46.
- [44] Evans T N L, Moore R H. The use of PDF/A in digital archives: a case study from archaeology[J]. International Journal of Digital Curation, 2014, 9(2):

123-138.

- [45] Ma Z, Wang N, Gao X, et al. From Reality to Perception: Genre-Based Neural Image Style Transfer[C]. in IJCAI, 2018, pp.3491-3497.
- [46] Booth D, Haas H, McCabe F, et al. Web services architecture, w3c working group note. vol, 2004, 2005: W3C.
- [47] Li T, Choi M, Lin L. Youtubestat. py: a python module to download youtube statistics and rankings. GitHub Repository, June, 2017.
- [48] Google. Chrome. <https://www.google.cn/intl/zh-CN/chrome/>, 2019-09-26.
- [49] D'souza S, Koehler H, Joshi A, et al. Quartz: time-as-a-service for coordination in geo-distributed systems. in Proceedings of the 4th ACM/IEEE Symposium on Edge Computing. 2019, pp.264-279.
- [50] 袁智, 李樾, 张正伟. 基于 HTML5 的跨平台家具网站设计与实现[J]. 信息记录材料, 2019 (11): 113.

致 谢

时光总是匆匆而过，在这短暂的三年时间中，研究生的这个大平台打开了我的视野，提高了我的眼界。这三年我逐渐明白什么是科研，也体会到了科研的艰辛与喜悦，科研是不断探索的过程，日复一日的科研生活不仅让我学习到了各种科学技术，也培养了我一种独立思考的能力和动手探索的习惯，这些能力和习惯在以后的生活工作中会让我受益颇多。

我特别要感谢我的导师何黎刚教授，自从我踏入研究生的门，何老师不经在学术上不厌其烦的指引我前进的方向，在生活上他也毫不吝啬的给予我关怀教会了我人生的道理，通过和他的谈话让我明白，我的生活可以过得更加的丰富多彩；而在学术上，何老师的一丝不苟的态度和渊博的学识深深地影响了我，研究生期间，我学过不少技术，做过不少项目，有时候心态也会出现浮躁和消沉，但何老师总能够抽出时间给予谆谆教诲，让我重重拾信心恢复良好的状态，并且在我写学术论文时，提出一些修改意见同时及时地调整了我的写作思路，让我的论文结构更加清晰。总的来说何老师这三年对我的帮助很大，我从他那里受益良多，他的学术态度和做人习惯给我留下了深刻的印象。特别是今年由于处在疫情期间导致我不能及时返校，何老师通过网络对我的论文进行了细致的指导。

在这里，我要感谢所有信息科学与工程学院的老师 and 朋友们，由于你们的存在，我们才能在一起学习上可以相互合作，生活上相互照顾，让我体会到了合作互助的力量，在迷茫的时候我们可以互相激励。我也要感谢教授们在课堂上精彩的授课，引导我走进了研究生的大门，给我提供了丰富的知识，为三年科研生活打下坚实的基础，同时多学科的学习开阔了我的视野。感谢在研究生期间结识的同学和朋友，你们的良好习惯和勤奋努力的学习态度促进着我不断地努力向前，这三年与你们同行给我的研究生生活增添了多种乐趣和色彩。我很荣幸能够在湖南大学这个“千年学府”中认真学习和快乐生活，在湖南大学的这段日子是我目前人生中最快乐的时光之一，对湖大提供的优质学习资源和良好生活条件表示感恩。

尤其要感谢的是我的父母，他们在我的生活上给予大量关怀，让我能够安心的在校学习，他们永远是我的港湾。

最后，还要感谢百忙之中对本论文进行评审的各位评委专家和教授，感谢他们对论文提出的宝贵意见。

金鹏

2020 年 6 月

附录 A 攻读硕士学位期间的学术成果

- [1] **Peng Jin**, Ligang He, Yilian Zhou, et al. MobiPDF: Reconstructing PDF contents for Mobile Devices[EB/OL]. Beijing:Sciencepaper Online[2020-05-29].
<http://www.paper.edu.cn/releasepaper/content/202005-216>.
- [2] **Peng Jin**, Ligang he, Yilian Zhou, et al. MobiPDF: Reconstructing PDF contents for Mobile Devices. IEEE Transactions on Pattern Analysis and Machine Intelligence (CCF A 类, Submitted, Under review).